UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

JOÃO VICTOR BRIGANTI DE OLIVEIRA

EXTENSÃO DO OPENMP PARA SUPORTE DE TÉCNICAS DE APROXIMAÇÃO

CAMPO MOURÃO

2025

JOÃO VICTOR BRIGANTI DE OLIVEIRA

EXTENSÃO DO OPENMP PARA SUPORTE DE TÉCNICAS DE APROXIMAÇÃO

OpenMP Extension for Approximate Computing Techniques

Proposta de Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Ciência da Computação do Curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.
Orientador(a): Prof. Dr. João Fabrício Filho

Coorientador(a): Prof. Dr. Rogério Aparecido Gonçalves

CAMPO MOURÃO

2025

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

Dedico este trabalho à minha família, pelos
momentos de ausência.

AGRADECIMENTOS

Primeiramente, agradeço a minha família, em especial ao meu pai e minha mãe, que foram meu grande apoio ao longo de toda essa jornada, sem os seus ensinamentos e apoio nada disso seria possível.

Agradeço aos meus orientadores Prof. Dr. João Fabrício Filho e ao Prof. Dr. Rogério Aparecido Gonçalves, pelas suas orientações e contribuições valiosas, não só neste trabalho como também ao longo de toda minha Iniciação Científica.

Também agradeço a minha namorada pela sua companhia e estar ao meu lado nos momentos dificéis.

Enfim, agradeço a todos que direta ou indiretamente me ajudaram e estiveram no meu lado durante minha caminhada.

“Nada lhe posso dar que já não exista em você mesmo. Não posso abrir-lhe outro mundo de imagens, além daquele que há em sua própria alma. Nada lhe posso dar a não ser a oportunidade, o impulso, a chave. Eu o ajudarei a tornar visível o seu próprio mundo, e isso é tudo.” (Hesse, 2025).

RESUMO

Com o fim da Lei de Moore e da escala de Dennard, diferentes técnicas computacionais surgiram para suprir a demanda por maior desempenho computacional e eficiência energética. A Computação Aproximada visa aumentar o desempenho e a eficiência energética de aplicações em troca de perda de precisão. A Computação Paralela busca obter maior desempenho por meio de um melhor aproveitamento dos processadores *multicore*. O `OpenMP` é uma ferramenta que facilita o desenvolvimento de aplicações paralelas por meio de anotações de código. O objetivo deste trabalho é implementar diferentes técnicas de aproximação no `OpenMP` por meio de modificações em sua infraestrutura no LLVM. As técnicas propostas são: perforação de laços, memoização aproximada, descarte de tarefas e relaxamento de ponto flutuante. As técnicas propostas serão avaliadas em um conjunto de sete aplicações tolerantes a aproximação, permitindo analisar o impacto dessas aproximações no desempenho e na precisão. Espera-se que os resultados obtidos evidenciem o potencial da combinação entre aproximação e paralelização para a otimização de aplicações.

Palavras-chave: paralelismo; llvm; alto desempenho; anotacões de código; computacão aproximada.

ABSTRACT

With the end of Moore's Law and Dennard scaling, different computational techniques have emerged to meet the demand for higher computational performance and energy efficiency. Approximate Computing aims to increase the performance and energy efficiency of applications in exchange for a loss of precision. Parallel Computing seeks to achieve higher performance through better utilization of multicore processors. `OpenMP` is a tool that facilitates the development of parallel applications through code annotations. The goal of this work is to implement different approximation techniques in `OpenMP` by modifying its infrastructure in LLVM. The proposed techniques are: loop perforation, approximate memoization, task dropping, and floating-point relaxation. The proposed techniques will be evaluated on a set of seven approximation-tolerant applications, allowing an analysis of the impact of these approximations on performance and accuracy. The results are expected to demonstrate the potential of combining approximation and parallelization for application optimization.

Keywords: parallelism; llvm; high performance; code annotations; approximate computing.

LISTA DE ILUSTRAÇÕES

Figura 1 – Modelo de execução do algoritmo de perforação de laço. **16**
Figura 2 – Erro introduzido na aritmética de ponto flutuante. **17**
Figura 3 – Arquitetura do `OpenMP`. **20**
Figura 4 – Modelo *fork-join* de execução do `OpenMP`. **22**
Figura 8 – Especificação do computador utilizado na avaliação experimental. **31**

LISTA DE TABELAS

Tabela 1 – Resumo e comparação entre os trabalhos citados. **24**
Tabela 2 – Representação da correlação entre as diferentes variáveis. **35**
Tabela 3 – Resumo das aplicações utilizadas nos experimentos **48**
Tabela 4 – Cronograma de execução das atividades **49**

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface*
CA	Computação Aproximada
CP	Computação Paralela
GPU	*Graphics Processing Unit*
HPC	*High-Performance Computing*
IIR	*Infinite Impulse Response*
MCR	*Miss-Classification Rate*
SIMD	*Single Instruction Multiple Data*
SMAPE	*Symmetric Mean Absolute Percentage Error*
SSIM	*Structural Similarity Index Measure*

SUMÁRIO

1 INTRODUÇÃO 11
1.1 Objetivos 12
1.2 Contribuições 12
1.3 Organização do Trabalho 13
2 FUNDAMENTAÇÃO TEÓRICA 14
2.1 Computação aproximada 14
2.1.1 Perforação de laço 15
2.1.2 Memoização 16
2.1.3 Aproximação de ponto flutuante 16
2.1.4 Descarte de tarefas 17
2.2 Impacto da Aproximação no Resultado de Computações 17
2.3 Computação paralela 18
2.4 Trabalhos Relacionados 21
3 PROPOSTA 25
3.1 Perforação de laço 25
3.2 Aproximação de ponto flutuante 26
3.3 Descarte de Tarefas 27
3.4 Memoização 29
4 MATERIAIS E MÉTODOS 31
4.1 Ambiente e Configuração Experimental 31
4.2 Conjunto de *Benchmarks* 32
4.2.1 2MM 32
4.2.2 Correlação 34
4.2.3 Deriche 36
4.2.4 Jacobi 2D 38
4.2.5 K-means 39
4.2.6 Mandelbrot 41
4.2.7 PI de Monte Carlo 44
4.3 Análise de Desempenho 46
4.4 Resumo do conjunto de aplicações 48
4.5 Cronograma 48
5 CONSIDERAÇÕES FINAIS 50
REFERÊNCIAS 51

1 INTRODUÇÃO

Por décadas, a Lei de Moore e a escala de Dennard impulsionaram melhorias exponenciais no desempenho computacional e na eficiência energética. A Lei de Moore permitiu aumentos constantes na densidade de transistores ao longo dos anos. Já a escala de Dennard, introduzida posteriormente, garantiu por um período que esses aumentos não viessem acompanhados de maior consumo de energia. Com o avanço tecnológico, cada uma dessas tendências atingiu seus limites, tornando os ganhos de desempenho cada vez mais difíceis, hoje limitados a somente 2% cento ao ano (Hennessy; Patterson, 2019). Mesmo com essas limitações, a crescente demanda por maior desempenho computacional e eficiência energética tem impulsionado inovações em uma ampla variedade de plataformas, desde sistemas embarcados até *datacenters*. Essa demanda é impulsionada pela crescente complexidade das cargas de trabalho modernas, como análise de dados em tempo real, aprendizado de máquina e processamento de sinais, que exigem vastos recursos computacionais (Mittal, 2016; Dalloo *et al.*, 2024).

Para atender a essas demandas sob restrições de desempenho e eficiência energia, pesquisadores têm explorado paradigmas alternativos de computação. A Computação Aproximada (CA) busca equilibrar precisão computacional e eficiência de recursos. CA parte do princípio de que muitas aplicações toleram resultados com imprecisão em limites aceitáveis, definidos pela percepção do usuário ou por métricas de qualidade específicas (Dalloo *et al.*, 2024). Ao explorar a diferença entre a precisão realmente exigida e a tradicionalmente garantida pelos sistemas, a CA viabiliza ganhos expressivos em desempenho, consumo de energia e utilização de *hardware* (Xu; Mytkowicz; Kim, 2016; Leon *et al.*, 2025b; Leon *et al.*, 2025a).

A Computação Paralela (CP) tornou-se um paradigma central na computação moderna devido à estagnação dos ganhos de desempenho por aumento de frequência dos processadores. Para continuar evoluindo, a indústria adotou arquiteturas *multicore*, nas quais múltiplas unidades de processamento trabalham simultaneamente. Nesse contexto, a CP é empregada para acelerar aplicações dividindo seu trabalho em partes que podem ser executadas em paralelo, reduzindo significativamente o tempo total de execução. No entanto, programar para essas arquiteturas não é trivial, o desenvolvedor precisa lidar com detalhes como criação e sincronização de *threads*, comunicação entre tarefas e gerenciamento de memória compartilhada. Ferramentas como o OpenMP surgiram justamente para abstrair essa complexidade, oferecendo um modelo de programação portátil baseado em diretivas que permitem paralelizar o código incrementalmente, sem comprometer a legibilidade, a manutenção ou a compatibilidade entre diferentes arquiteturas (Gonçalves *et al.*, 2016; Alrawais, 2021). Assim, o OpenMP consolidou-se como uma das principais ferramentas de paralelização, especialmente em ambientes de Computação de Alto Desempenho (*High-Performance Computing*, HPC), onde simulações científicas e aplicações numéricas dependem fortemente de execução eficiente (Adhikari *et al.*, 2012).

Embora as técnicas de CA ofereçam benefícios expressivos ao permitir que aplicações troquem precisão por desempenho ou eficiência energética, sua adoção eficaz exige identificar cuidadosamente as regiões de código nas quais a aproximação é segura e vantajosa. Essa análise envolve compreender o comportamento da aplicação, seus requisitos de qualidade e o impacto que imprecisões controladas podem ter em seus resultados (Sampson *et al.*, 2015; Reis; Wanner, 2021). Este trabalho parte da hipótese de que é possível combinar CP e CA harmoniosamente, explorando o paralelismo de plataformas modernas enquanto se introduz flexibilidade no nível de precisão. A integração dessas abordagens, especialmente quando apoiada por ferramentas portáveis e amplamente adotadas como o OpenMP, pode ampliar os ganhos de desempenho sem sacrificar a facilidade de uso e a compatibilidade entre diferentes sistemas.

1.1 Objetivos

O objetivo geral deste trabalho é melhorar o desempenho de aplicações de *High-Performance Computing* (HPC), por meio da combinação de técnicas de aproximação e paralelização.

Para atingir esse objetivo, foram definidos os seguintes objetivos específicos:

- Implementar uma extensão do `OpenMP` com suporte a técnicas de aproximação baseadas em anotações;
- Desenvolver um conjunto de aplicações voltado à avaliação de técnicas de CA;
- Adicionar suporte em tempo de execução para múltiplos algoritmos de aproximação no `OpenMP`;
- Integrar um passo de perforação de laços ao *pipeline* de otimização do LLVM;
- Avaliar experimentalmente as técnicas propostas em um ambiente de CA.

1.2 Contribuições

As principais contribuições deste trabalho são: a implementação de uma extensão do `OpenMP` com suporte a técnicas de aproximação e a integração dessas funcionalidades à infraestrutura do LLVM. A extensão proposta exigiu modificações na infraestrutura do LLVM, bem como adaptações em algoritmos e escalonadores do `OpenMP`, a fim de possibilitar o suporte a tarefas aproximadas. Resultados parciais deste trabalho foram publicados na Escola Região de Alto Desempenho (ERAD-SP) (Oliveira; Gonçalves; Filho, 2024b), Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD-2024) (Oliveira; Gonçalves; Filho, 2024a) e no Seminário de Iniciação Científica e Tecnológica da UTFPR (SICITE 2024) (Oliveira; Gonçalves; Filho, 2024c).

1.3 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma: no Capítulo 2 são apresentados os conceitos fundamentais de CA e CP relacionados a este estudo, incluindo as técnicas de aproximação abordadas, perforação de laço, memoização aproximada, descarte de tarefas e relaxamento de ponto flutuante, além de uma introdução ao `OpenMP` e uma revisão de trabalhos correlatos. No Capítulo 3 é apresentada a proposta dos construtores implementados no `OpenMP` utilizando a infraestrutura do LLVM. No Capítulo 4 são detalhadas a metodologia, as ferramentas empregadas para a realização dos experimentos e o cronograma de execução deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos relacionados a CA e CP. Primeiramente aborda-se a definição de CA, seu uso em aplicações, os desafios enfrentados por essa estratégia e o detalhamento dos algoritmos de perforação de laço, memoização, aproximação de ponto flutuante e o descarte de tarefas. Por fim apresenta também conceitos de CP com um foco no uso da ferramenta OpenMP.

2.1 Computação aproximada

A CA consiste em um conjunto de técnicas aplicadas a cenários onde os resultados exatos não são estritamente necessários ou em que as aplicações apresentam resiliência suficiente para lidar com pequenas imprecisões em suas computações. Nessas situações, o uso de tais técnicas pode proporcionar ganhos significativos de desempenho e eficiência energética, mantendo a margem de erro em níveis reduzidos. Um exemplo é o uso de redes neurais para aproximar a divergência de *branch* em instruções *Single Instruction Multiple Data* (SIMD), capaz de alcançar aceleração de até 14,8$\times$ com acurácia de 96% (Grigorian; Reinman, 2015). Por esse motivo, a CA se mostra especialmente atraente em domínios como análise de dados, reconhecimento de padrões, processamento de imagens e sinais (Mittal, 2016; Chippa *et al.*, 2013).

As diferentes estratégias de aproximação podem ser classificadas conforme a camada em que são aplicadas na pilha computacional. No nível de *hardware*, destacam-se técnicas que envolvem desde unidades funcionais aproximadas, como somadores, multiplicadores e divisores, até abordagens como *overclocking* (Leon *et al.*, 2025b; Leon *et al.*, 2025a) e o uso de memórias aproximadas (Fabrício *et al.*, 2020). No nível de *software*, encontram-se estratégias mais generalistas, incluindo linguagens de programação aproximada (Sampson *et al.*, 2015), *runtimes* com suporte à aproximação (Li; Park; Mahlke, 2018; Reis; Wanner; Rigo, 2024) e otimizações realizadas em nível de compilador (Oliveira; Gonçalves; Filho, 2024b; Oliveira; Gonçalves; Filho, 2024a).

Apesar de seus benefícios, esse paradigma enfrenta desafios significativos. Em primeiro lugar, nem todas as aplicações são resilientes a erros em sua execução, limitando o uso de técnicas mais generalistas e torna necessário definir métricas que permitam ajustar o grau de precisão desejado, seja pelo usuário ou pela própria aplicação (Mittal, 2016). Além disso, medir corretamente o nível de erro introduzido não é trivial, pois nem sempre está claro qual métrica deve ser aplicada à saída da aplicação (Felzmann *et al.*, 2021). Outro ponto crítico é a propagação dos erros, que pode comprometer toda a execução: em alguns casos, a aplicação pode falhar abruptamente; em outros, concluir sua execução de forma aparentemente correta, mas com resultados significativamente distantes do esperado (Fabrício *et al.*, 2020).

Diante disso é possível observar que a CA não se restringe a um único método, mas a um conjunto de estratégias que exploram diferentes aspectos da execução de um programa. Na

subseção seguinte, são detalhados quatro dessas estratégias: perforação de laço, memoização, aproximação de ponto flutuante e o descarte de tarefas.

2.1.1 Perforação de laço

A perforação de laço é uma das técnicas mais simples de computação aproximada, cuja ideia central consiste em omitir determinadas iterações de um laço, reduzindo assim o tempo de execução de uma região de código (Sidiroglou-douskos *et al.*, 2011).

Essa técnica, pode ser implementada em tempo de compilação ou dinamicamente o que permite ajustar o grau de omissão em tempo de execução (Li; Park; Mahlke, 2018).

Em ambos os casos é comum implementar essa otimização em laços que estão em um formato canônico (Openmp, 2018). A Listagem 1 apresenta a estrutura de um laço canônico, geralmente expresso na forma de um `for`, com três elementos principais:

- **Expressão inicial:** define o limite inferior do laço, ou seja, o ponto em que a computação começa.
- **Expressão de teste:** geralmente um operador relacional que compara a variável de indução com o limite superior, delimitando o espaço de iterações.
- **Expressão de incremento/decremento:** atualiza a variável de indução e define o passo do laço.

Listagem 1 – Estrutura de um laço canônico.

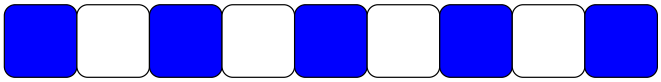
```
1        for (int i = 0; i < N; i += a) {
2            ...
3        }
4
```

Fonte: Autoria própria (2025).

Uma técnica comumente encontrada na implementação desse algoritmo é a de incrementar a expressão de incremento, se considerarmos isso no Listagem 1 teríamos que as iterações deixariam de ser `a` em `a` vezes, teríamos um incremento de `a + 1` vezes. Se considerarmos que inicialmente tem o valor `a = 1` ele passaria a ser `a = 2` e o comportamento do laço seguiria estando na Figura 1, nela temos que os blocos em branco seriam iterações não executadas enquanto os blocos em azul seriam iterações executadas.

Uma forma simples de aplicar a perforação de laço é alterar o passo do incremento da variável de indução, de modo que parte das iterações seja ignorada. No exemplo da Listagem 1, se inicialmente o incremento for `a = 1` (`i++`), todas as iterações serão executadas. Entretanto, ao modificar o incremento para `a = 2` (`i += 2`), somente metade delas será realizada, já que o laço passa a “pular” uma iteração a cada passo. Esse comportamento é ilustrado na Figura 1, na qual blocos em azul representam iterações executadas e blocos em branco representam iterações omitidas.

Figura 1 – Modelo de execução do algoritmo de perforação de laço.



Fonte: Autoria própria (2025).

2.1.2 Memoização

A memoização é tradicionalmente uma técnica associada à programação dinâmica que utiliza uma *cache* de resultados para acelerar a execução de determinadas computações. Seu princípio consiste em armazenar os resultados de chamadas já realizadas, indexados pelas entradas correspondentes. Assim, quando a função é invocada novamente com os mesmos parâmetros, o valor previamente calculado é retornado diretamente, evitando a repetição da computação (Michie, 1968).

A memoização aproximada, também chamada de memoização temporal, estende esse conceito ao considerar tempo de execução e localização como critérios adicionais. Sua ideia principal é explorar que chamadas consecutivas a uma mesma função tendem a produzir resultados similares (Tziantzioulis; Hardavellas; Campanoni, 2018).

Na prática, as saídas são armazenadas em uma estrutura de dados e reutilizadas seletivamente em chamadas subsequentes. As estratégias de *cache* podem variar, incluindo modelos globais, específicos por função ou mesmo dependentes de contexto. O reuso é normalmente controlado por um limiar de tolerância, que avalia a variação das saídas: se os resultados permanecerem estáveis dentro desse limite, a função é considerada estável e, portanto, pode ser memoizada (Tziantzioulis; Hardavellas; Campanoni, 2018).

2.1.3 Aproximação de ponto flutuante

A representação de números reais por meio de números de ponto flutuante envolvem aproximação por padrão devido à impossibilidade de representar valores contínuos com memória finita, se impossibilita representar um número infinito com um número finito de *bits* (Monniaux, 2008). Por esse motivo aritmética com números de ponto flutuante usualmente introduzem erros de aproximação que variam conforme a ordem de execução dessas operações, resultando em um valor completamente diferente do esperado. Um exemplo disso pode ser observado na Figura 2.

Para reduzir este tipo de problema, compiladores usualmente evitam aplicar certas otimizações que podem amplificar o erro nesse tipo de operação. Porém, alguns compiladores como é o caso do `Clang` e o `GCC`, suportam a opção `-fast-math` que permite a aplicação dessas otimizações em todo um módulo de compilação (GNU Project, 2004; The LLVM Project, 2025). E de maneira similar o `MSVC` suporta o uso da diretiva de compilação `#pragma float_control` para definir regiões do código que permitem o uso dessas otimizações em certas regiões decódigo (Microsoft, 2021).

Figura 2 – Erro introduzido na aritmética de ponto flutuante.

```
a = 1e9+1; b = -1e9; c = 0.1;
(a + b) + c
> 1.1
a + (b + c)
> 1.100000023841858
```

Fonte: Autoria própria (2025).

2.1.4 Descarte de tarefas

O descarte de tarefas é um conceito abrangente que engloba diferentes técnicas de aproximação, incluindo, em alguns casos, a própria perforação de laço. Sua ideia central baseia-se em computações que podem ser divididas em unidades menores, chamadas tarefas, das quais uma parte delas é descartada (Mittal, 2016).

As implementações dessa técnica costumam adotar uma taxa de descarte, que define a fração de tarefas a serem eliminadas durante a execução do programa. A Figura 1 também pode ser utilizada para ilustrar esse mecanismo: os blocos brancos representam tarefas descartadas, enquanto os blocos azuis indicam as tarefas efetivamente executadas.

2.2 Impacto da Aproximação no Resultado de Computações

A aplicação de técnicas de aproximação em ambientes computacionais introduz, inerentemente, erros nas saídas dessas computações. Esses erros podem gerar diferentes consequências para a aplicação, variando desde falhas explícitas até erros silenciosos, como a corrupção de dados (Fabrício, 2022). Parte dos estudos em CA busca lidar justamente com essas falhas silenciosas, pois, embora resultem em saídas imprecisas, ainda podem gerar resultados úteis, diferentemente das falhas diretas, que usualmente não geram nenhuma saída.

A CA busca estabelecer um equilíbrio entre o nível de aproximação aplicado e as falhas resultantes, de modo que o usuário final sofra o menor impacto possível na precisão dos resultados, enquanto se obtém um ganho significativo no uso de recursos computacionais. Entretanto, um dos desafios enfrentados por essas técnicas está na dificuldade de caracterizar como as aproximações afetam o resultado, especialmente quando se consideram aplicações com naturezas e requisitos de precisão distintos.

Neste trabalho, são utilizadas métricas de qualidade para diferenciar e comparar as saídas aproximadas em relação às saídas exatas. As métricas foram escolhidas com base do que é encontrado na literatura, e conforme o tipo de cada aplicação. No entanto, a relação entre a qualidade numérica e a aceitabilidade prática de uma aplicação não é necessariamente direta. Existem casos, em que uma alta qualidade pode não ser útil no contexto final da aplicação, enquanto saídas de qualidade inferior ainda podem ser consideradas aceitáveis (Felzmann *et*

al., 2021), um exemplo disso seria uma imagem com muito ruído, apesar de sua qualidade ruim, em certos contextos ela continua sendo aceitável. Dessa forma, a análise baseada somente em métricas de qualidade pode levar a interpretações incorretas sobre a utilidade real dos resultados.

Para avaliar a perda de acurácia das aplicações, foram adotadas três métricas distintas: o *Symmetric Mean Absolute Percentage Error* (SMAPE), o *Miss-Classification Rate* (MCR) e o *Structural Similarity Index Measure* (SSIM), definidos, respectivamente, nas Equações Equação 1, Equação 2 e Equação 3.

$$SMAPE(A, F) = \frac{100}{n} \sum_{t=1}^{n} \frac{|F_t - A_t|}{|A_t| + |F_t|} \tag{1}$$

$$MCR(A, F) = \frac{1}{n} \sum_{t=1}^{n} \iota(A_t \neq F_t) \tag{2}$$

Em que as variáveis são definidas como:

- n: número de elementos no *array* de dados;
- A_t: t-ésimo elemento obtido pela execução acurada;
- F_t: t-ésimo elemento obtido pela execução aproximada;
- ι: função indicadora que retorna 1 quando a condição é verdadeira e 0 caso contrário.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1) \cdot (2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1) \cdot (\sigma_x^2 + \sigma_y^2 + c_2)} \tag{3}$$

Em que as variáveis são definidas como:

- μ_x: média dos *pixels* da imagem x;
- μ_y: média dos *pixels* da imagem y;
- σ_x: variância dos *pixels* da imagem x;
- σ_y: variância dos *pixels* da imagem y;
- σ_{xy}: covariância entre as imagens x e y;
- c_1 e c_2: constantes pequenas utilizadas para evitar divisão por zero.

2.3 Computação paralela

A CP é uma área da computação que busca explorar arquiteturas modernas, cada vez mais baseadas em processadores *multicore*. Esse conceito pode ser melhor compreendido a partir da taxonomia de Flynn (Flynn, 1972), que classifica as arquiteturas de computadores de acordo com o fluxo de instruções e de dados:

- ***Single Instruction, Single Data stream* (SISD):** Instrução única que atua sobre uma única fonte de dados.
- ***Single Instruction, Multiple Data stream* (SIMD):** Instrução única aplicada simultaneamente a múltiplos dados.
- ***Multiple Instructions, Single Data stream* (MISD):** Múltiplas instruções atuando sobre uma mesma fonte de dados.
- ***Multiple Instructions, Multiple Data stream* (MIMD):** Múltiplas instruções atuando em paralelo sobre múltiplas fontes de dados.

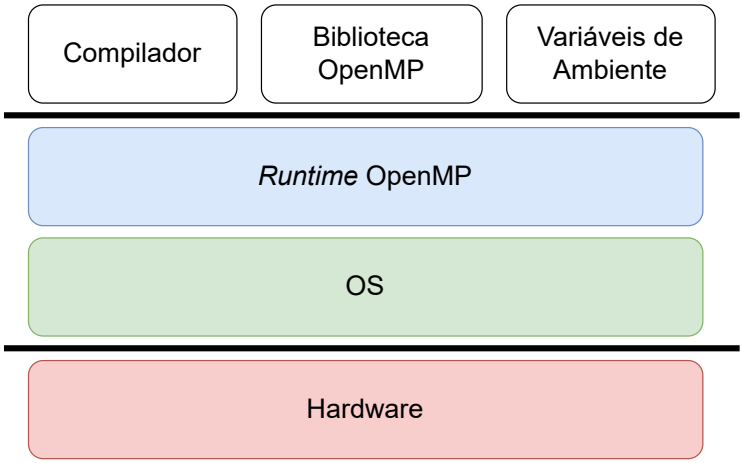
Dentro desse modelo, o paralelismo pode ser explorado de diferentes maneiras. No nível do processador, técnicas como *pipeline*, predição de desvios (*branch prediction*) e escalonamento dinâmico permitem dividir o código em pequenas unidades que podem ser executadas simultaneamente.

Algumas arquiteturas foram além e incorporaram extensões SIMD, permitindo a utilização de processadores vetoriais capazes de realizar a mesma operação sobre múltiplos elementos de um vetor unidimensional em um único ciclo de instrução. Em cenários mais especializados, como nas *Graphics Processing Unit*s (GPUs), esse modelo é expandido para múltiplas dimensões de dados, possibilitando que milhares de operações sejam executadas em paralelo e tornando-as altamente adequadas para cargas de trabalho massivamente paralelizáveis (Hennessy; Patterson,). Outro mecanismo fundamental é o uso de *threads*, que representam abstrações de execução em nível de software, permitindo que diferentes unidades computacionais operem concorrentemente em múltiplos processadores. Cada *thread* possui seu próprio contexto de execução, mas pode compartilhar recursos como memória e dispositivos de entrada/saída, viabilizando desde o paralelismo de tarefas independentes até a cooperação em algoritmos que exploram granularidades mais finas (Tanenbaum, ; Hennessy; Patterson,).

É nesse contexto que surge o OpenMP, uma *Application Programming Interface* (API) projetada para estender C, C++ e Fortran com suporte à paralelização de aplicações por meio de anotações de código. Essas anotações especificam o como aquela região de código deve ser executada. Seu modelo de programação permite explorar automaticamente diferentes formas de paralelismo, seja pelo *offloading* para GPUs, pelo uso de múltiplas *threads*, ou ainda pela vetorização via SIMD, aproveitando as unidades vetoriais das arquiteturas modernas (Mattson; He; Koniges, 2019).

O OpenMP é estruturado em uma arquitetura em camadas, composta por anotações de código, suporte da *runtime* e configurações do ambiente de execução, conforme ilustrado na Figura 3. Esse modelo permite ao programador controlar diversos aspectos da execução por meio de diretivas e chamadas à biblioteca, como a definição do número de *threads* em uma região paralela e a escolha do comportamento do escalonador. Dessa forma, o OpenMP oferece um controle flexível sobre o paralelismo do programa.

Figura 3 – Arquitetura do `OpenMP`.



Fonte: Autoria própria (2025).

A Listagem 2 apresenta a implementação sequencial de um produto escalar entre vetores. A paralelização utilizando a biblioteca `pthreads`, apresentada na Listagem 3, exige a criação explícita de *threads*, gerenciamento de regiões de memória e sincronização dos resultados parciais. Em contraste, a Listagem 4 implementado com o OpenMP, evidencia a simplicidade proporcionada pelas diretivas: uma única anotação é suficiente para paralelizar o laço de cálculo, delegando à *runtime* a criação e sincronização das *threads*.

Listagem 2 – Estrutura de um laço canônico.

```
1        for (int i = 0; i < N; i++) {
2            result += a[i] * b[i];
3        }
4
```

Fonte: Autoria própria (2025).

Tipicamente, uma diretiva OpenMP é aplicada a um laço para indicar que ele pode ser executado em paralelo. Durante a compilação, esse bloco de código é isolado e transformado em tarefas, enquanto a *runtime* prepara as estruturas necessárias para o modelo *fork-join*. Por exemplo, a diretiva `#pragma omp parallel` cria um time de *threads*, cada uma executando uma cópia do código anotado. O compilador gerencia variáveis privadas, memória de pilha e pontos de sincronização automaticamente. Ao final, as *threads* são reunificadas (*joined*), e a execução retorna ao fluxo sequencial. Essa abordagem permite paralelizar regiões de código com esforço reduzido de gerenciamento, mantendo controle sobre sincronização e compartilhamento de dados (Mattson; He; Koniges, 2019).

O modelo *fork-join* pode ser observado de forma esquemática na Figura 4. Nele, a *runtime* cria múltiplas *threads* para a execução paralela, sincronizadas ao final, restando apenas a *thread* principal para continuar a execução sequencial. O OpenMP também suporta regiões paralelas aninhadas, possibilitando hierarquias de paralelismo.

Listagem 3 – Estrutura de um laço canônico.

```
1        void *compute_partial_dot(void *arg) {
2            ThreadArgs *args = (ThreadArgs *)arg;
3            args->partial_sum = 0.0;
4
5            for (int i = args->start; i < args->end; i++) {
6                args->partial_sum = 0.0; += args->a[i] * args->b[i];
7            }
8
9            return NULL;
10       }
11
12   ...
13
14   pthread_t threads[NUM_THREADS];
15   ThreadArgs args[NUM_THREADS];
16   result = 0.0;
17   chunk_size = N / NUM_THREADS;
18
19   for (int i = 0; i < NUM_THREADS; i++) {
20       args[i].start = i * chunk_size;
21       args[i].end = (i == NUM_THREADS - 1) ? N : (i + 1) * chunk_size;
22       args[i].a = a;
23       args[i].b = b;
24       pthread_create(&threads[i], NULL, compute_partial_dot, &args[i]);
25   }
26
27   for (int i = 0; i < NUM_THREADS; i++) {
28       pthread_join(threads[i], NULL);
29       result += args[i].partial_sum;
30   }
31
```

Fonte: Autoria própria (2025).

Listagem 4 – Estrutura de um laço canônico.

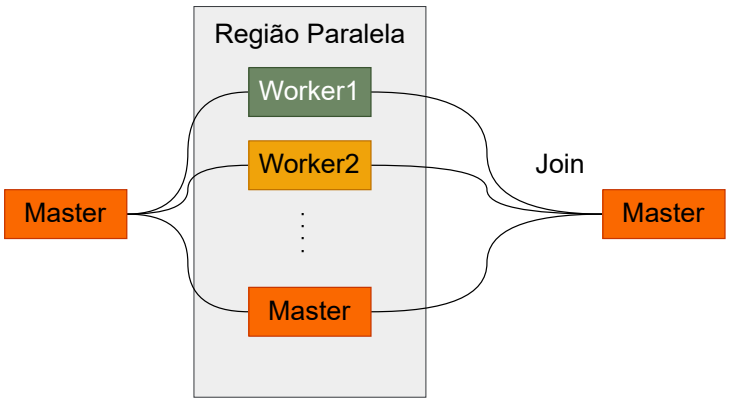
```
1        #pragma omp parallel for reduction(+ : result)
2        for (int i = 0; i < N; i++) {
3            result += a[i] * b[i];
4        }
5
```

Fonte: Autoria própria (2025).

2.4 Trabalhos Relacionados

Esta seção apresenta arcabouços baseados no paradigma de CA que implementam suporte ao nível de linguagem de programação, especialmente aqueles que utilizam anotações de código e mecanismos de execução em *runtime*. Os arcabouços e aplicações selecionados buscam oferecer ao desenvolvedor maior controle sobre as técnicas utilizadas e sobre as regiões de código que podem ser aproximadas.

Figura 4 – Modelo *fork-join* de execução do `OpenMP`.



Fonte: Autoria própria (2025).

O ACCEPT (Sampson *et al.*, 2015) (Approximate C Compiler for Energy and Performance Trade-Off) é um arcabouço implementado sobre a infraestrutura do LLVM, que introduz a palavra-chave `ACCEPT` na sintaxe das linguagens C/C++. Essa palavra-chave permite diferenciar regiões de código aproximadas daquelas que devem ser executadas de forma precisa. O arcabouço também implementa etapas de análise para verificar a possibilidade de aproximação dessas regiões, avaliando não apenas a validade das transformações, mas também viabilizando a aplicação de diferentes técnicas de otimização aproximada, de maneira análoga ao funcionamento de um compilador paralelizador.

A infraestrutura do ACCEPT permite, por meio de anotações de código e análises estáticas e dinâmicas, identificar regiões de código potencialmente aproximáveis. Essas regiões podem ser exploradas para aplicar técnicas como perforação de laço, elisão de sincronização e até mesmo o uso de redes neurais em *hardware* para estimar saídas. Além disso, o arcabouço incorpora um *autotuner* que utiliza métricas fornecidas pelo usuário para avaliar se as aproximações aplicadas mantêm a qualidade de saída nos limites esperados, garantindo que o programa não apresente falhas durante a execução.

Vassiliadis *et al.* (2015) propõem uma extensão ao OpenMP que incorpora suporte à CA no modelo de programação baseado em tarefas. Em sua proposta, o programador pode especificar funções aproximadas alternativas que podem ser escolhidas em tempo de execução para substituir versões precisas de determinadas tarefas. Assim, parte das tarefas é executada de forma exata, enquanto outras utilizam versões aproximadas, reduzindo o consumo energético e mantendo a qualidade de acordo com limites definidos pelo usuário.

O arcabouço também introduz um mecanismo de barreira para garantir que uma fração mínima das execuções alcance o grau de acurácia desejado. Para gerenciar essa execução, são propostas duas políticas no runtime: *Global Task Buffering* (GTB), que toma decisões informadas a partir da análise conjunta de todas tarefas pendentes, e a *Local Queue History* (LQH), baseada no histórico local de filas, em que cada *worker* decide de maneira autônoma o nível de aproximação com base nas execuções anteriores.

Lashgar, Atoofian e Baniasadi (2018) propõem a integração da técnica de perforação de laço ao `OpenACC` para melhorar o desempenho, executando somente um subconjunto das iterações do laço. Dessa forma, parte das saídas é calculada de maneira exata, enquanto o restante é aproximado. Para mitigar os impactos dessa abordagem na precisão e no tempo de execução dentro do `OpenACC`, os autores introduzem um mecanismo de correção implementado diretamente no kernel. Esse mecanismo identifica as saídas ausentes e aplica estratégias como a cópia ou a média dos valores vizinhos já computados, a fim de estimar as entradas faltantes.

O HPAC (Parasyris *et al.*, 2021) é um arcabouço de técnicas de computação aproximada voltadas para aplicações HPC, implementado sobre a infraestrutura do LLVM. Ele utiliza anotações de código `pragma` para C/C++, permitindo a definição de regiões de código aproximadas, e se destaca por sua interoperabilidade com o `OpenMP`. As técnicas principais incluem perforação de laço e memoização aproximada, e o arcabouço fornece ainda uma ferramenta de análise que permite ao usuário anotar múltiplas regiões de código e avaliar a qualidade dos resultados obtidos para cada configuração.

O HPAC-Offload (Fink *et al.*, 2023) é uma extensão do HPAC (Parasyris *et al.*, 2021) voltada para aplicações em GPU, que adapta sua arquitetura para lidar com as limitações do modelo de memória e paralelismo dessas plataformas. Assim como no HPAC, as técnicas implementadas incluem perforação de laço e memoização aproximada, mas com um gerenciamento de memória consciente da GPU: os dados internos são armazenados na memória compartilhada do bloco, permitindo reutilização entre threads ativas sem sobrecarregar a memória global. O arcabouço também define níveis hierárquicos de aproximação (*thread*, *warp* e bloco) para evitar divergência e *deadlocks*.

Em contraste com outros trabalhos, nossa abordagem baseia-se na extensão do próprio framework `OpenMP`, modificando sua *runtime* e alterando suas diretivas de compilação para oferecer suporte nativo a técnicas de CA. Ao incorporar o controle de aproximação diretamente no `OpenMP`, habilitamos o escalonamento e a execução de tarefas sem comprometer sua API. Essa integração simplifica a adoção de CA em CP, como também abre caminho para uma exploração mais ampla e eficaz da aproximação em aplicações que já utilizam o OpenMP. A Tabela 1 apresenta um breve resumo e comparação entre todos os trabalhos citados.

Tabela 1 – Resumo e comparação entre os trabalhos citados.

Trabalho	**Linguagem / Infraestrutura**	**Suporte a Paralelismo**	**Técnicas de Aproximação**	**Diferenciais**
ACCEPT (Sampson *et al.*, 2015)	C/C++, LLVM	Sequencial e paralelo	Perforação de laço, elisão de sincronização e redes neurais em *hardware*	Validação de aproximações em tempo de execução; Aplicação de múltiplas técnicas de otimização
Vassiliadis et al. (Vassiliadis *et al.*, 2015)	C/C++, OpenMP	Paralelismo baseado em tarefas	Funções aproximadas	Controle de acurácia; Decisões globais e locais para aproximação
Lashgar et al. (Lashgar; Atoofian; Baniasadi, 2018)	C/C++, OpenACC	Paralelismo em laço	Perforação de laço	Estima valores ausentes para manter precisão em laços aproximados
HPAC (Parasyris *et al.*, 2021)	C/C++, LLVM, OpenMP	Regiões paralelas	Perforação de laço, memoização aproximada	Interoperabilidade com OpenMP; Análise de múltiplas regiões de código
HPAC-Offload (Fink *et al.*, 2023)	C/C++, LLVM, OpenMP	Paralelismo via GPU	Perforação de laço, memoização aproximada	Interoperabilidade com OpenMP; Hierarquia de aproximação em GPU
Este Trabalho	**C/C++, LLVM, OpenMP**	**Regiões paralelas**	**Perforação de laço, memoização aproximada, aproximação em ponto flutuante, descarte de tarefas**	**Integração direta com OpenMP; Combina análise em tempo de compilação e decisões em tempo de execução; Aplica múltiplas técnicas de aproximação visando desempenho**

Fonte: Autoria própria (2025).

3 PROPOSTA

Este trabalho apresenta um construtor para o `OpenMP` que facilita a implementação de algoritmos de aproximação. Assim como outros construtores da API, a proposta utiliza anotações de código, para identificar regiões que podem ser executadas de forma aproximada. A integração dessas anotações ao arcabouço do `OpenMP` assegura que as técnicas sejam portáteis e facilmente adaptáveis a diferentes trechos de código.

A Listagem 5 apresenta a anotação `pragma approx`, que delimita uma região de código a ser aproximada e específica a técnica a ser aplicada. Múltiplas cláusulas podem ser combinadas em um único construtor para indicar diferentes parâmetros de aproximação. Entretanto, a implementação atual suporta somente uma técnica de aproximação por vez.

Listagem 5 – Sintaxe do construtor `approx`.

```
1        #pragma omp approx [clause, [[,]clause]]
2
```

Fonte: Autoria própria (2025).

As próximas seções apresentam as técnicas implementadas e suas respectivas cláusulas e parâmetros.

3.1 Perforação de laço

A cláusula `perfo` habilita o uso de perforação de laços (*loop perforation*), permitindo reduzir o número de iterações executadas. Essa cláusula deve ser utilizada em conjunto com a cláusula `for`, e suporta modificadores que definem tanto o algoritmo de perforação quanto a taxa de descarte. Os modificadores disponíveis são: `init`, `fini`, `small`, `large` e `default`. A Listagem 6 apresenta a sintaxe da cláusula, tanto o tipo da perforação quanto a taxa de descarte são parâmetros obrigatórios nessa cláusula.

Listagem 6 – Sintaxe da cláusula `perfo`

```
1        #pragma omp approx perfo({init, fini, small, large, default}, \
2                                 drop-rate)
3
```

Fonte: Autoria própria (2025).

O modificador `init` descarta a quantidade especificada de iterações no início do laço. Esse comportamento é implementado por meio de uma alteração na variável de indução, que ajusta o limite inferior do laço para cada *thread*. Dessa forma, quando a *thread* assume sua fatia do laço, as primeiras iterações dessa fatia são descartadas. O comportamento dessa cláusula é apresentado na Figura 5a.

O modificador `fini` descarta as iterações finais do laço, de modo análogo ao `init`, afetando também os limites do laço por *thread*. Seu comportamento é apresentado na Figura 5b.

O modificador `small` aplica uma estratégia probabilística, na qual um valor aleatório é usado para decidir o descarte de iterações, com base em uma condição de módulo envolvendo a taxa de descarte. Se a condição do valor aleatório ser igual à taxa for satisfeita, a variável de indução é incrementada pela taxa de descarte, efetivamente pulando algumas iterações, como mostra a Figura 5c.

A Equação 4, apresenta a fórmula usada para verificar a condição. As variáveis envolvidas nessa fórmula são:

- ξ: Número aleatório.
- ub: Limite superior do laço.
- lb: Limite inferior do laço.
- d: Taxa de descarte.

$$\xi \mod ((ub - lb) + ub) \mod d \tag{4}$$

O modificador `large` aplica uma perforação mais agressiva, incrementando diretamente a variável de indução pela taxa de descarte. Essa substituição é feita em tempo de execução, interceptando a lógica padrão de incremento ou decremento e substituindo-a por uma chamada de função que ajusta a variável de indução conforme a taxa. A Figura 5d apresenta o seu comportamento no laço, os quadrados em azul são iterações executadas, enquanto os quadrados brancos são iterações que não foram executadas.

O modificador `default` apresenta comportamento semelhante ao `large`, porém a taxa de descarte é aplicada em tempo de compilação, eliminando a sobrecarga associada a chamadas adicionais durante a execução.

Figura 5 – Características de cada modificador.

(a) Comportamento do modificador `init`.

(b) Comportamento do modificador `fini`.

(c) Comportamento do modificador `small`.

(d) Comportamento do modificador `large`.

Fonte: Autoria própria (2025).

3.2 Aproximação de ponto flutuante

A cláusula `fastmath` habilita otimizações de ponto flutuante, seguindo a semântica do parâmetro de compilação `-fast-math` do compilador `Clang` (The LLVM Project, 2025).

Quando aplicada, o compilador extrai a região de código anotada e adiciona metadados indicando que essa região permite otimizações agressivas de ponto flutuante.

A principal diferença entre os dois métodos é que a cláusula `fastmath` oferece controle mais granular, permitindo que as otimizações sejam aplicadas apenas na região especificada. Em contraste, o parâmetro global `-fast-math` força a aplicação dessas otimizações em todo o módulo de compilação. A Listagem 7 apresenta a sintaxe da cláusula.

Listagem 7 – Sintaxe da cláusula `fastmath`

```
1        #pragma omp approx fastmath
2
```

Fonte: Autoria própria (2025).

Uma característica fundamental dessa cláusula está nas otimizações relacionadas à associatividade das operações de ponto flutuante. Isso permite que o compilador reordene operações de modo a melhorar o desempenho. Embora a reordenação seja válida para números reais, na aritmética de ponto flutuante ela pode produzir resultados numericamente distintos devido aos erros de arredondamento acumulados em cada operação.

A Listagem 8 apresenta um exemplo prático do uso da cláusula. A função *sqrt*, por exemplo, normalmente requer verificações adicionais para lidar com entradas inválidas, como valores negativos ou *Not a Number* (*NaN*). Ao aplicar `fastmath`, o compilador assume que essas entradas não ocorrerão, emitindo diretamente a instrução de hardware *sqrt*, sem checagens extras, o que reduz a sobrecarga e melhora o desempenho.

Listagem 8 – Sintaxe da cláusula `fastmath`.

```
1        float sqrt_approx(float x)
2        {
3            #pragma omp approx fastmath
4            {
5                return sqrt(x);
6            }
7        }
8
```

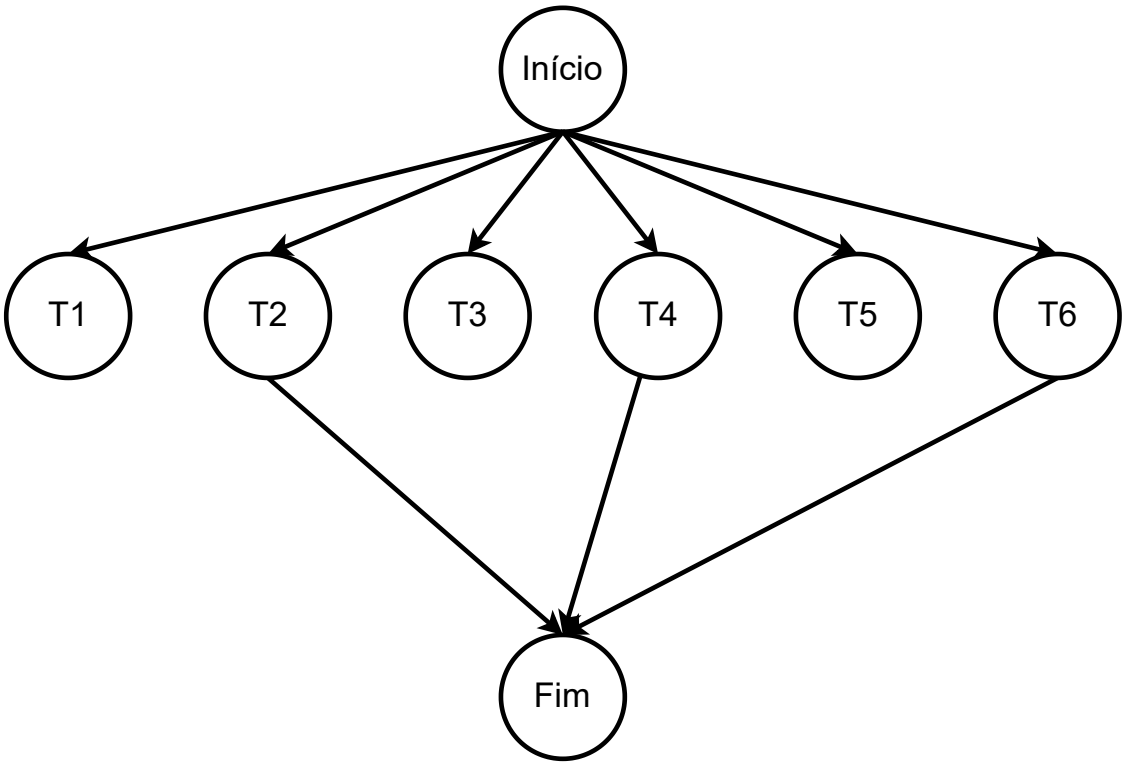
Fonte: Autoria própria (2025).

3.3 Descarte de Tarefas

O conceito de tarefas no `OpenMP` permite a execução assíncrona de regiões de código. Quando uma *thread* encontra uma região anotada com a cláusula `task`, um objeto de tarefa é criado para ser executado posteriormente. Essa tarefa é então escalonada dinamicamente, podendo ser executada por qualquer outra *thread*. Esse modelo de execução é particularmente útil em computações heterogêneas e irregulares, pois permite uma distribuição dinâmica da carga de trabalho entre as *threads* (Ayguadé *et al.*, 2007; OpenMP Architecture Review Board, 2021).

O construtor `taskloop` estende esse conceito para laços. Cada laço é particionado em pedaços, definidos pelo parâmetro `grainsize`, e uma tarefa é criada para cada pedaço. Essas tarefas podem então ser executadas de forma independente por diferentes *threads*.

Figura 6 – Modelo de execução da cláusula `drop`.



Fonte: Autoria própria (2025).

Neste trabalho, propomos a cláusula `drop`, que pode ser usada em conjunto com `taskloop` para implementar uma estratégia de aproximação baseada no descarte de tarefas. De forma similar à técnica de perforação de laço, essa cláusula permite que o usuário especifique uma taxa de descarte, que define quantas tarefas serão eliminadas. A Equação 4 é utilizada para determinar quais tarefas devem ser descartadas.

A Figura 6 ilustra esse processo, e a Listagem 9 apresenta a sintaxe da cláusula.

Listagem 9 – Sintaxe da cláusula `drop`

```
1        #pragma omp approx taskloop drop(drop-rate)
2
```

Fonte: Autoria própria (2025).

O Listagem 10 mostra um exemplo de uso da cláusula `drop` em conjunto com `taskloop` para aproximar o resultado de uma soma. o descarte de tarefas pode aumentar o desempenho da aplicação, pois ao ignorar determinadas execuções, a *thread* fica com sua execução livre para avançar mais rapidamente para outras regiões de código.

Listagem 10 – Exemplo de uso da cláusula `drop`.

```
1    #pragma omp parallel
2    {
3        #pragma omp single
4        {
5            #pragma omp approx taskloop drop(3) reduction(+ : sum)
6            {
7                for (int i = 0; i < N; i++) {
8                    sum += values[i];
9                }
10           }
11       }
12   }
13
```

Fonte: Autoria própria (2025).

3.4 Memoização

A cláusula `memo` habilita otimizações de memoização aproximada, permitindo o reuso de resultados previamente calculados. Essa estratégia é particularmente útil para funções onde a relação entre entrada e saída se mantém estável ao longo do tempo, desde que a execução da mesma região de código produza resultados consistentes, independentemente das entradas específicas.

A Listagem 11 apresenta a sintaxe da cláusula `memo`. Essa cláusula deve ser utilizada em conjunto com a diretiva `shared` para garantir que, em tempo de execução, as variáveis indicadas possam ser acessadas. Opcionalmente, o modificador `threshold` pode ser especificado para definir um limite aceitável de erro entre os resultados que serão reutilizados.

Listagem 11 – Sintaxe da cláusula `memo`.

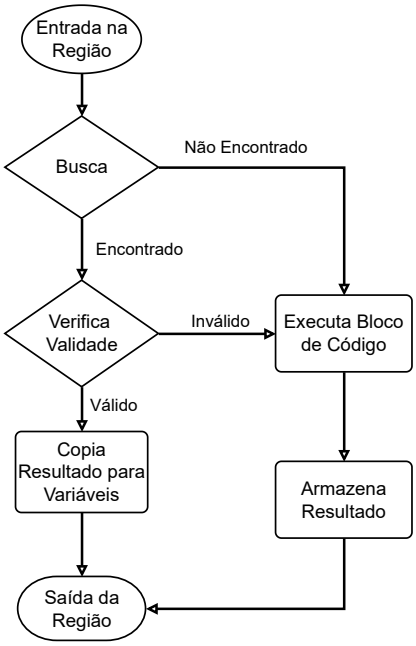
```
1        #pragma omp approx memo shared([variables]) [threshold(error-limit)]
2
```

Fonte: Autoria própria (2025).

Internamente, a técnica é implementada usando uma árvore AVL, cujos nós folha são estruturas *hash map*. Em tempo de execução, as chamadas à função interceptam a região de código anotada e verificam se existe algum resultado previamente armazenado válido para aquela região. Se houver um resultado válido, ele é reutilizado; caso contrário, o bloco de código é executado normalmente e o resultado é armazenado para uso futuro. O fluxo de execução está ilustrado na Figura 7. É importante ressaltar que, nesta versão da implementação, a verificação de validade é feita exclusivamente com base na região de código, sem considerar diretamente os valores de entrada. Portanto, o reuso é determinado apenas pela execução da mesma região, independentemente das entradas fornecidas.

Quando o usuário especifica um limiar por meio do modificador `threshold`, a execução incluir uma verificação que mede a diferença relativa entre os resultados da região. Se

Figura 7 – Modelo de execução da cláusula `memo`.



Fonte: Autoria própria (2025).

essa diferença estiver dentro do limite definido, o valor é armazenado e considerado válido para reuso. Caso nenhum limiar seja especificado, os valores previamente calculados são sempre reutilizados, sem verificação adicional. O Listagem 12 apresenta um exemplo de uso da cláusula `memo` aplicado a um laço, usando um limiar de 10%. Com a memoização habilitada, execuções repetidas dessa região de código podem ser substituídas por valores previamente calculados, respeitando o limite de erro definido.

Listagem 12 – Exemplo de uso da cláusula `memo`.

```
1        #pragma omp approx memo shared(sum) threshold(0.1)
2        {
3            for (int i = 0; i < N; i++) {
4                sum += values[i];
5            }
6        }
7
```

Fonte: Autoria própria (2025).

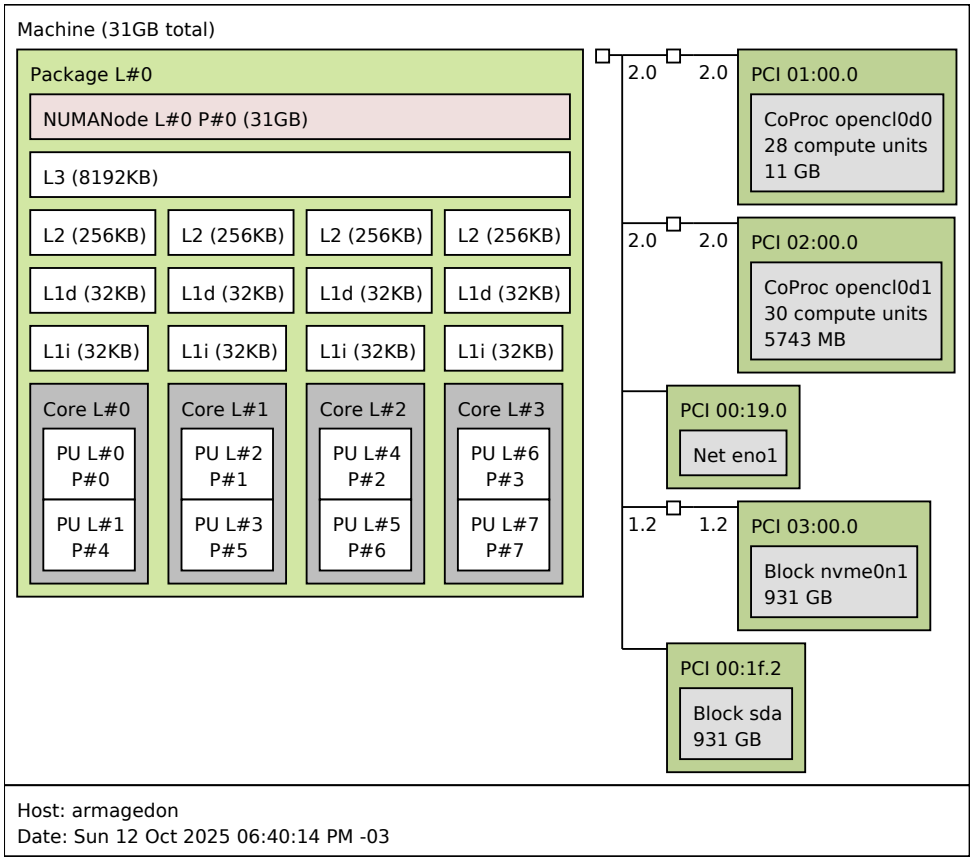
4 MATERIAIS E MÉTODOS

Este capítulo apresenta a metodologia e os recursos utilizados no desenvolvimento deste trabalho. A seção 4.1 descreve as configurações de *hardware* das máquinas empregadas na execução dos experimentos. Em seguida, a seção 4.2 detalha as aplicações selecionadas para os testes, bem como as técnicas de otimização adotadas e o posicionamento das regiões paralelas. A seção 4.3 aborda os métodos utilizados para a coleta e análise dos dados de desempenho. Por fim, a seção 4.4 apresenta uma síntese de todos os experimentos realizados, organizada em formato tabular.

4.1 Ambiente e Configuração Experimental

Todos os experimentos deste trabalho foram realizados em um computador equipado com um processador Intel Core i7-4690, com frequência base de 3.6 GHz, quatro núcleos físicos e suporte a oito *threads*. A hierarquia de memória *cache* apresenta 32 KB, 256 KB e 8 MB nos níveis L1, L2 e L3, respectivamente. O sistema possui 32 GB de memória RAM DDR3, distribuídos em quatro módulos de 8 GB configurados em *dual-channel*, cada um operando a 1600 MT/s. O sistema operacional utilizado foi o Ubuntu 22.04 LTS. A Figura 8 apresenta a hierarquia de memória e a disposição dos núcleos do processador utilizados nos testes.

Figura 8 – Especificação do computador utilizado em testes.



Fonte: Autoria própria (2025).

Cada aplicação foi executada com 1, 2, 4 e 8 *threads* em 10 execuções por configuração para incluir variações no ambiente, todas compiladas com *flag* de compilação `-O3` (nível de maior otimização do código gerado pelo compilador). Nos algoritmos que envolvem aleatoriedade, como `perfo` e `drop`, foi utilizada uma semente fixa para garantir a reprodutibilidade dos resultados. As técnicas `perfo`, `drop` e `memo` foram testadas com cinco níveis de taxa de exclusão: 0.1, 0.2, 0.3, 0.4 e 0.5. Esses valores foram escolhidos por representarem níveis progressivos de descarte, variando de 10% a 50% da computação, permitindo observar de maneira controlada como cada aplicação reage a aumentos graduais de aproximação e identificar o ponto onde o impacto no desempenho ou na qualidade se torna mais significativo.

4.2 Conjunto de *Benchmarks*

A seleção das aplicações que compõem o conjunto de benchmarks teve como principais critérios a tolerância a erros e a estrutura do algoritmo. Buscou-se contemplar uma gama ampla de domínios, incluindo *machine learning*, álgebra linear, modelos estatísticos e multimídia. Outro objetivo foi preservar ao máximo o código original durante a inserção das regiões paralelizadas, evitando modificações significativas na implementação. As aplicações 2MM, Correlação, Deriche e Jacobi 2D foram obtidas a partir do conjunto PolyBench (Pouchet, 2011), K-Means foi obtida a partir do Rodinia (Che *et al.*, 2009) e o Mandelbrot e PI de Monte Carlo foram adaptações do Debian Benchmarks Game (Debian Project, 2018).

4.2.1 2MM

2mm (Pouchet, 2011) é uma aplicação que realiza a multiplicação sequencial de duas matrizes, utilizando três matrizes distintas: A, B e C. O processo de execução é ilustrado pelas Equações 5 e 6.

$$D = A \cdot B \tag{5}$$

$$E = C \cdot D \tag{6}$$

O algoritmo de multiplicação de matrizes empregado segue o padrão clássico (Axler, 2015) descrito na Equação 7, que mostra a multiplicação de duas matrizes 2x2, representando o processo de somas e multiplicações sequenciais. A implementação utilizada pelo *benchmark* é apresentada no Algoritmo 1.

$$\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} \cdot \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \end{bmatrix} = \begin{bmatrix} a_1b_1 + a_2b_3 & a_1b_2 + a_2b_4 \\ a_3b_1 + a_4b_3 & a_3b_2 + a_4b_4 \end{bmatrix} \tag{7}$$

Algoritmo 1 – Multiplicação de matrizes.

requer Matrizes A de tamanho $n \times m$, B de tamanho $m \times p$, C de tamanho $n \times p$
1: **para** $i = 0$ até n, com passo 1 **faça**
2: **para** $j = 0$ até p, com passo 1 **faça**
3: **para** $k = 0$ até m, com passo 1 **faça**
4: $C[i][j] \leftarrow C[i][j] + A[i][k] \cdot B[k][j]$
5: **finaliza para**
6: **finaliza para**
7: **finaliza para**

Fonte: Adaptado do Pouchet (2011).

Nesta aplicação, ambas as multiplicações são executadas em sequência, conforme as Equações 5 e 6. Para reduzir o *overhead*, optou-se por utilizar uma única região paralela que abrange ambas multiplicações. Além da paralelização foi empregado a técnica de *loop tiling* (Bakos, 2016). Essa técnica consiste em dividir o espaço de iteração em blocos menores, *tiles*, de modo que cada bloco seja completamente processado antes de avançar para o próximo. Diferentemente da iteração convencional, que percorre toda uma dimensão por vez, no *tiling* os acessos a memória são reorganizados para melhorar a localidade espacial e temporal.

O principal benefício dessa otimização está no uso mais eficiente da *cache*. Idealmente, cada bloco deve ser dimensionado de forma que seus dados caibam inteiramente na *cache*, permitindo que todo o bloco seja carregado e processado de uma só vez. Dessa forma, reduz-se o número de acessos à memória principal e evita-se o recarregamento de dados já utilizados, aumentando o desempenho da aplicação (Bakos, 2016). O Algoritmo 2 apresenta o pseudocódigo de implementação dessa técnica na multiplicação de matrizes.

Algoritmo 2 – Multiplicação de matrizes com *loop tiling*.

requer Matrizes A, B e C de tamanho $n \times n$
requer Tamanho do bloco bs
1: **para** $ii = 0$ até n, com passo bs **faça**
2: **para** $kk = 0$ até n, com passo bs **faça**
3: **para** $i = ii$ até $\min(ii + bs, n)$ **faça**
4: **para** $k = kk$ até $\min(kk + bs, n)$ **faça**
5: $a_val \leftarrow A[i][k]$
6: **para** $j = 0$ até n, com passo 1 **faça**
7: $C[i][j] \leftarrow C[i][j] + a_val \times B[k][j]$
8: **finaliza para**
9: **finaliza para**
10: **finaliza para**
11: **finaliza para**
12: **finaliza para**

Fonte: Autoria própria (2025).

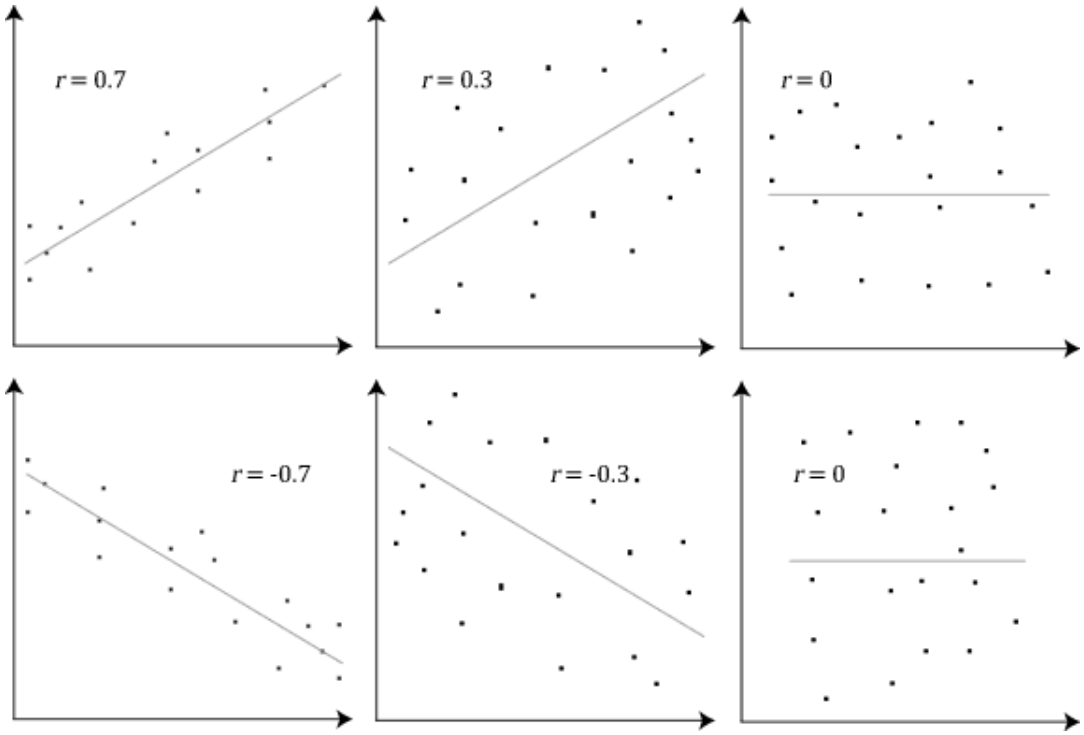
Outra técnica empregada, também utilizada nas demais aplicações, foi a alocação contínua de memória. Em vez de alocar um bloco de memória separado para cada linha da matriz, foi alocado um único bloco contíguo. Esta técnica foi adotada visando a redução de chamadas de alocação e melhor da localidade espacial, aumentando o desempenho.

4.2.2 Correlação

A correlação é uma medida estatística que expressa o grau de associação entre duas variáveis (Morettin; Bussab, 2010). Quando há correlação entre duas variáveis, isso indica que mudanças em uma delas tendem estar relacionadas a mudanças na outra.

A correlação é positiva quando, ambas as variáveis aumentam ou diminuem juntas, ou negativa, quando uma variável aumenta enquanto a outra diminui. O valor da correlação varia de -1 a 1. Valores próximos de 1 indicam uma forte correlação positiva, enquanto valores próximos de -1, indicam uma forte correlação negativa e valores próximos de 0 indicam uma fraca ou nenhuma correlação. A Figura 9, ilustra esses três tipos de correlação, nos quais é possível observar visualmente o comportamento das variáveis em cada caso.

Figura 9 – Gráfico do cálculo de coeficiente de correlação de Pearson.



Fonte: Statistics (2019).

A fórmula comumente utilizada para calcular a correlação é chamada de *Pearson Correlation Formula*, apresentada na Equação 8 (Morettin; Bussab, 2010). Nessa equação, x e y representam os valores das variáveis, enquanto m_x e m_y correspondem as suas respectivas médias. A implementação utilizada pelo Polybench (Pouchet, 2011) é apresentada nos Algoritmos 3 e 4, os quais respectivamente fazem a construção da matriz de correlação e o cálculo do coeficiente.

$$r = \frac{\sum(x - m_x)(y - m_y)}{\sqrt{\sum(x - m_x)^2 \sum(y - m_y)^2}} \tag{8}$$

O cálculo da correlação entre as diferentes variáveis é apresentado na Tabela 2, como uma matriz simétrica, na qual os valores localizados na parte inferior da matriz são iguais aos da

Algoritmo 3 – Construção da matriz de correlação.

requer Matrizes C e D de tamanho $n \times m$
1: **para** $i = 0$ até n, com passo 1 **faça**
2: **para** $j = 0$ até n, com passo 1 **faça**
3: **se** (j == i) **então**
4: $r \leftarrow 1.0$
5: **senão,**
6: $r \leftarrow \texttt{correlation}(\texttt{linha}(D, i), \texttt{linha}(D, j), m)$
7: **finaliza se**
8: $C[i][j] \leftarrow r$
9: $C[j][i] \leftarrow r$
10: **finaliza para**
11: **finaliza para**

Fonte: Adaptado do Pouchet (2011).

Algoritmo 4 – Cálculo do coeficiente de correlação de Pearson.

requer *Array* X e Y de tamanho m
1: $sumX \leftarrow 0$
2: $sumY \leftarrow 0$
3: $sumXY \leftarrow 0$
4: $sumX2 \leftarrow 0$
5: $sumY2 \leftarrow 0$
6: **para** $i = 0$ até n, em passos de 1 **faça**
7: $\texttt{sumX} \leftarrow \texttt{sumX} + X[i]$
8: $\texttt{sumY} \leftarrow \texttt{sumY} + Y[i]$
9: $\texttt{sumXY} \leftarrow \texttt{sumXY} + X[i] * Y[i]$
10: $\texttt{sumX2} \leftarrow \texttt{sumX2} + X[i] * X[i]$
11: $\texttt{sumY2} \leftarrow \texttt{sumY2} + Y[i] * Y[i]$
12: **finaliza para**

Fonte: Adaptado do Pouchet (2011).

parte superior em relação à diagonal principal. Essa diagonal principal contém valores iguais a 1.0, correspondentes a autocorrelação perfeita de cada variável. Essa simetria permite otimizar o cálculo, reduzindo o número de iterações, uma vez que é suficiente computar os coeficientes para somente uma das metades da matriz.

Tabela 2 – Representação da correlação entre as diferentes variáveis.

	a	**b**	**c**	**d**
a	1.0	ab	ac	ad
b	ab	1.0	bc	bd
c	ac	bc	1.0	cd
d	ad	bd	cd	1.0

Fonte: Autoria própria (2025).

No Algoritmo 3 essa propriedade de simétrica é explorada, e é também nessa região de código, mais especificamente em seu laço mais externo, que foram adicionados as anotações de paralelização, permitindo que o cálculo dos coeficientes fossem executados concorrentemente.

4.2.3 Deriche

Algoritmo 5 – Algoritmo Deriche de suavização.

requer Imagem I_{in} de tamanho $h \times w$
requer Coeficiente de suavização α
1: $I_{out1} \leftarrow$ nova Matriz $h \times w$
2: $I_{out2} \leftarrow$ nova Matriz $h \times w$
3: $I_{final} \leftarrow$ nova Matriz $h \times w$
{Cálculo dos coeficientes de suavização}
4: $k \leftarrow \frac{(1-e^{-\alpha})^2}{1+2\alpha e^{-\alpha}-e^{-2\alpha}}$
5: $a_1 \leftarrow k$
6: $a_2 \leftarrow k \cdot e^{-\alpha} \cdot (\alpha - 1)$
7: $a_3 \leftarrow k \cdot e^{-\alpha} \cdot (\alpha + 1)$
8: $a_4 \leftarrow -k \cdot e^{-2\alpha}$
9: $b_1 \leftarrow 2 \cdot e^{-\alpha}$
10: $b_2 \leftarrow -e^{-2\alpha}$
{Varredura horizontal (esquerda para direita)}
11: **para** $i = 0$ até $h - 1$, com passo 1 **faça**
12: $y_{m1} \leftarrow 0$, $y_{m2} \leftarrow 0$
13: **para** $j = 0$ até $w - 1$, com passo 1 **faça**
14: $I_{out1}[i, j] \leftarrow a_1 \cdot I_{in}[i, j] + a_2 \cdot I_{in}[i, j - 1] + b_1 \cdot y_{m1} + b_2 \cdot y_{m2}$
15: $y_{m2} \leftarrow y_{m1}$
16: $y_{m1} \leftarrow I_{out1}[i, j]$
17: **finaliza para**
18: **finaliza para**
{Varredura horizontal (direita para esquerda)}
19: **para** $i = 0$ até $h - 1$, com passo 1 **faça**
20: $y_{p1} \leftarrow 0$, $y_{p2} \leftarrow 0$
21: **para** $j = w - 1$ até 0, com passo -1 **faça**
22: $I_{out2}[i, j] \leftarrow a_3 \cdot I_{in}[i, j + 1] + a_4 \cdot I_{in}[i, j + 2] + b_1 \cdot y_{p1} + b_2 \cdot y_{p2}$
23: $y_{p2} \leftarrow y_{p1}$
24: $y_{p1} \leftarrow I_{out2}[i, j]$
25: **finaliza para**
26: **finaliza para**
{Combinação final}
27: $I_{final} \leftarrow I_{out1} + I_{out2}$
28: **retorna** I_{final}

Fonte: Adaptado do Pouchet (2011).

Em processamento de imagens, o filtro de Deriche é um algoritmo de filtragem recursiva utilizado tanto para suavização de imagens quanto para detecção de bordas (Deriche, 1987). Este algoritmo foi desenvolvido como uma otimização do detector de bordas de Canny, buscando aproximar as propriedades de um filtro Gaussiano, por meio de um filtro *Infinite Impulse Response* (IIR).

Intuitivamente, o filtro de Deriche funciona de maneira diferente de um filtro baseado em uma janela fixa de *pixels*. Nesse algoritmo, os resultados intermediários de *pixels* anteriores são armazenados e utilizados no cálculo dos *pixels* seguintes. O filtro percorre a imagem duas vezes, primeiramente na horizontal, da esquerda para a direita, e depois da direita para

a esquerda, aplicando recursivamente os valores anteriores. Esse processo resulta em uma suavização eficiente, mantendo contornos suaves nas bordas da figura.

O nível dessa suavização é controlado pelo parâmetro α, que ajusta a supressão de ruído e a precisão na localização da borda. Esse parâmetro define os coeficientes a e b, sendo utilizados tanto nas Equações 9 e 10, sendo aplicados respectivamente da esquerda para a direita e da direita para a esquerda.

$$y^1_{ij} = a_1 x_{ij} + a_2 x_{ij-1} + b_1 y^1_{ij-1} + b_2 y^1_{ij-2} \tag{9}$$

$$y^2_{ij} = a_3 x_{ij+1} + a_4 x_{ij+2} + b_1 y^2_{ij+1} + b_2 y^2_{ij+2} \tag{10}$$

O filtro implementado segue o padrão do Polybench (Pouchet, 2011), somente a suavização foi implementada, não há uma geração das bordas da imagem. O Algoritmo 5 apresenta a implementação da suavização, na alteração desse *benchmark* foi adicionado uma região paralela que engloba todos os laços do algoritmo.

A aplicação desse filtro com um $\alpha = 1.5$ em uma imagem pode ser observado na Figura 10, na qual a Figura 10a é a imagem original em preto e branco e Figura 10b a imagem com o filtro aplicado. Na imagem, o filtro suaviza as bordas, tornando mais simples o processo detectá-las.

Figura 10 – Aplicação do filtro de Deriche.

(a) Imagem original **(b) Imagem com Deriche aplicado**

Fonte: Autoria própria (2025).

4.2.4 Jacobi 2D

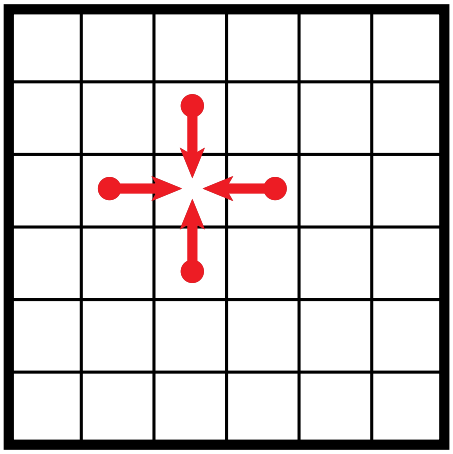
Em análise numérica, o método de Jacobi é um algoritmo iterativo utilizado para resolver sistemas de equações lineares do tipo $Ax = b$ (Szép, 2007). Para que o método funcione adequadamente, o sistema deve atender a algumas condições específicas:

- **Matriz quadrada:** A matriz dos coeficientes deve ter o mesmo número de linhas e colunas;
- **Sistema linear:** Não deve haver termos como multiplicação entre variáveis (xy), potências superiores a 1 (x^2, y^3, etc.), nem funções não lineares, como seno, cosseno ou exponenciais (e^x);
- **Diagonal dominante:** Embora não seja uma exigência absoluta, a presença de dominância diagonal na matriz dos coeficientes é altamente recomendada, pois contribui para a garantia de convergência do método. Em sua ausência, o processo iterativo pode apresentar convergência lenta ou até mesmo não convergir.

Esse algoritmo foi selecionado do conjunto de Polybench (Pouchet, 2011) e adotado neste trabalho por sua natureza iterativa e previsível em termos de comportamento computacional. O processo inicia-se com a atribuição de valores iniciais arbitrários ao vetor x. Em seguida, realiza-se uma iteração para calcular uma nova estimativa da solução, com base exclusivamente nos valores da iteração anterior. Esses novos valores são utilizados como entrada para a próxima iteração. Esse procedimento é repetido até que a diferença entre iterações consecutivas seja inferior a um limiar de tolerância pré-definido, indicando a convergência para uma solução estável.

O Jacobi 2D é uma aplicação do método de Jacobi adaptado para problemas discretizados em duas dimensões. Sua implementação se baseia em um algoritmo de estêncil, no

Figura 11 – Execução do algoritmo de estêncil.



Fonte: Gentryx (2011).

qual cada ponto de uma malha bidimensional é atualizado com base na média artimética dos seus quatro vizinhos imediatos (acima, abaixo, esquerda e a direita), como mostra a Figura 11. Após um número suficiente de iterações, a malha converge para uma configuração estável que representa uma aproximação da solução contínua do problema. O Algoritmo 6 apresenta a implementação utilizada pelo *benchmark*.

Algoritmo 6 – Método de Jacobi em duas dimensões

requer Matriz A e B de tamanho $n \times n$
requer Quantidade de passos do algoritmo p
1: **para** $t = 0$ até $p - 1$, com passo 1 **faça**
2: **para** $i = 1$ até $n - 1$, com passo 1 **faça**
3: **para** $j = 1$ até $n - 1$, com passo 1 **faça**
4: $B[i][j] \leftarrow (A[i][j] + A[i][j+1] + A[i][j-1] + A[i-1][j] + A[i+1][j])/4$
5: **finaliza para**
6: **finaliza para**
7: **para** $i = 1$ até $n - 1$, com passo 1 **faça**
8: **para** $j = 1$ até $n - 1$, com passo 1 **faça**
9: $A[i][j] \leftarrow (B[i][j] + B[i][j+1] + B[i][j-1] + B[i-1][j] + B[i+1][j])/4$
10: **finaliza para**
11: **finaliza para**
12: **finaliza para**

Fonte: Adaptado do Pouchet (2011).

O Algoritmo 6 apresenta dependência sequencial entre as etapas de atualização, dificultando a paralelização total. Para contornar isso, a paralelização foi aplicada no laço mais externo, permitindo que cada etapa de varredura seja executada em paralelo. Já os laços internos foram anotados com diretivas de sincronização, assegurando que os dados produzidos em uma etapa estejam disponíveis antes da próxima atualização.

4.2.5 K-means

O algoritmo K-means (Macqueen, 1967) é uma técnica de agrupamento, cujo objetivo é particionar um conjunto de dados em k grupos, de forma que os dados num mesmo grupo de modo que eles sejam semelhantes entre si.

A ideia central do algoritmo consiste em definir inicialmente k centróides, que são pontos de referência no espaço de amostragem. Em seguida, cada ponto de dado é associado ao centróide mais próximo, com base em uma medida de distância, usualmente a distância euclidiana. Após essa etapa de atribuição, o algoritmo calcula a média dos pontos atribuídos a cada grupo e atualiza a posição dos centróides com base nesses valores. A Figura 12 ilustra o agrupamento dos dados ao longo das iterações do algoritmo K-means. Na figura, os triângulos representam os centróides, os pontos correspondem aos dados e as regiões coloridas indicam as áreas que delimitam os agrupamentos formados.

O processo de agrupamento é repetido iterativamente, os dados são reagrupados com base nos novos centróides e os centróides são novamente recalculados. O algoritmo continua

Algoritmo 7 – Algoritmo K-means

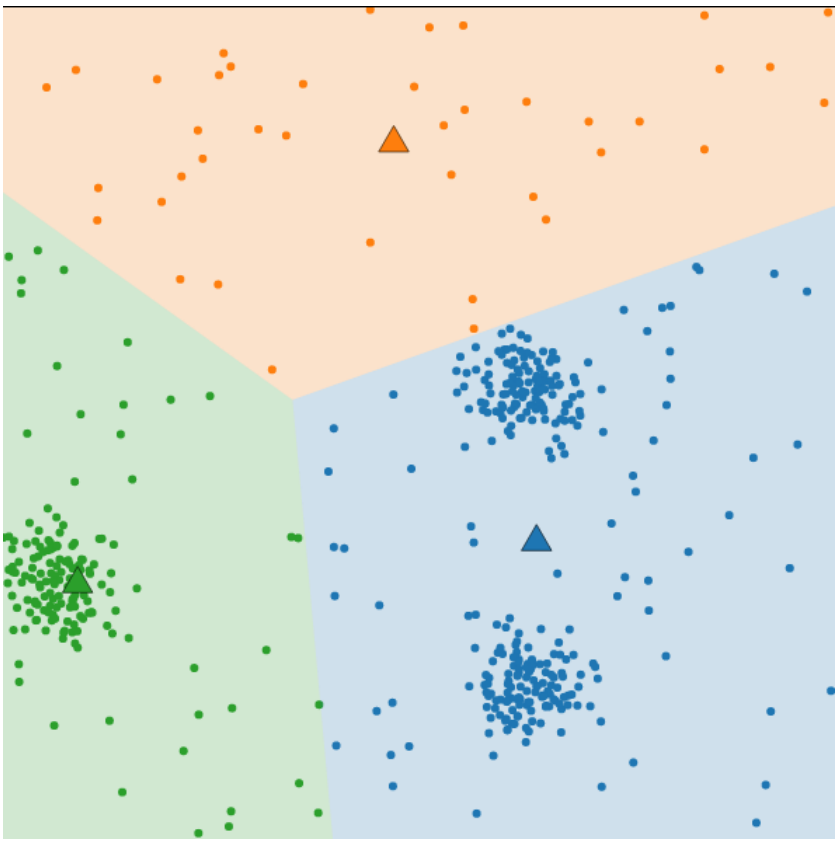
requer Matriz $features$ de dimensão $numPoints \times numFeatures$
requer $numClusters$, número de $clusters$ que serão formados
requer $iterations$, quantidade de iterações realizada pelo algoritmo
requer $threshold$, limiar de parada do algoritmo
1: Matriz $centroids$ de tamanho $numClusters \times numFeatures$, iniciado com 0
2: Matriz $newCentroids$ de tamanho $numClusters \times numFeatures$, iniciado com 0
3: *Array* $membership$ de tamanho $numPoints$, iniciado com -1
4: *Array* $newCentroidsLen$ de tamanho $numClusters$, iniciado com 0
5: **para** $i = 0$ até $numClusters$, com passo 1 **faça**
6: $n \leftarrow$ índice aleatório entre 0 e $numPoints - 1$
7: **para** $j = 0$ até $numFeatures$, com passo 1 **faça**
8: *centroids*$[i][j] \leftarrow$ *features*$[n][j]$
9: **finaliza para**
10: **finaliza para**
11: **para** $iter = 0$ até $iterations$ **faça**
12: *delta* $\leftarrow 0$
13: **para** $j = 0$ até $numPoints$ **faça**
14: $index \leftarrow \texttt{find_nearest_point}(centroids, features[j], numFeatures)$
15: **se** $membership[j] \neq index$ **então**
16: $delta \leftarrow delta + 1$
17: **finaliza se**
18: $membership[j] \leftarrow index$
19: $newCentroidsLen[index] \leftarrow newCentroidsLen[index] + 1$
20: **para** $k = 0$ até $numFeatures$ **faça**
21: $newCentroids[index][k] \leftarrow newCentroids[index][k] + features[j][k]$
22: **finaliza para**
23: **finaliza para**
24: **para** $j = 0$ até $numClusters$ **faça**
25: **se** $newCentroidsLen[j] > 0$ **então**
26: **para** $k = 0$ até $numFeatures$ **faça**
27: $centroids[j][k] \leftarrow newCentroids[j][k]/newCentroidsLen[j]$
28: **finaliza para**
29: **finaliza se**
30: **finaliza para**
31: Zerar $newCentroids$
32: Zerar $newCentroidsLen$
33: **se** $delta \leq threshold$ **então**
34: $break$
35: **finaliza se**
36: **finaliza para**
37: **retorna** $centroids$

Fonte: Adaptado do Pouchet (2011).

até atingir um critério de parada, que pode ser um número fixo de iterações ou a estabilização dos centróides.

O Algoritmo 7 (Che *et al.*, 2009) apresenta a implementação desse algoritmo, que reflete essa lógica detalhadamente, incluindo a etapa de inicialização dos centróides, a atribuição de pontos, o recálculo dos centróides e a verificação do critério de parada.

Figura 12 – Agrupamentos ao longo das iterações do algoritmo K-means



Fonte: Incheol (2015).

Durante a execução, o algoritmo também avalia a variância, uma medida que indica o quão disperso os pontos estão em relação ao seu centróide. Quanto menor a variância, mais compactos são os grupos formados. Embora o objetivo não seja minimizar a variância até zero, uma variância excessivamente alta pode indicar que os dados não estão sendo agrupados de maneira eficiente.

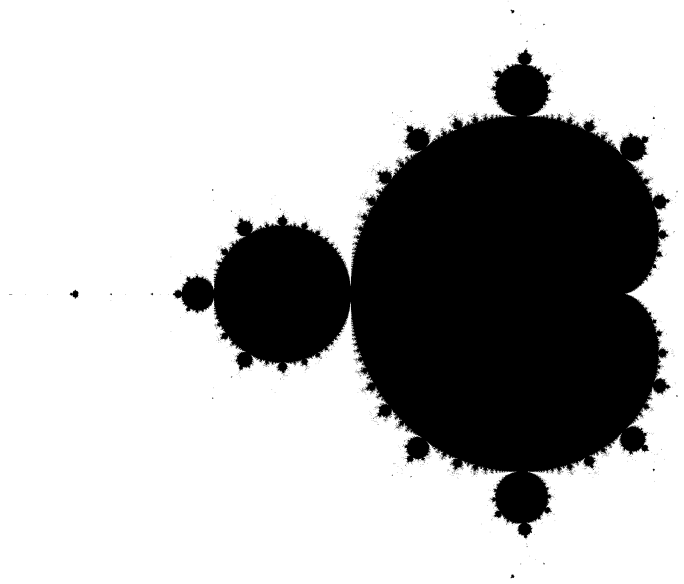
Na implementação apresentada no Algoritmo 7, a paralelização foi introduzida por meio de anotações dos laços internos, responsáveis pelas maiores cargas computacionais. Em particular, foram paralelizados os laços que percorrem os pontos de dados (*numPoints*) e o laço que percorre os agrupamentos (*numclusters*).

4.2.6 Mandelbrot

Mandelbrot é um conjunto de duas dimensões, definidas no conjunto dos números complexos. Ele é construído a partir da iteração da função apresentada na Equação 11, na qual z e c são números complexos, com c constante para cada ponto avaliado e a condição inicial $z_0 = 0$ (Devaney, 1999).

$$z_{n+1} = z_n^2 + c \tag{11}$$

Figura 13 – Representação do conjunto Mandelbrot.



Fonte: Autoria própria (2025).

Nem todos os valores de c pertencem ao conjunto de Mandelbrot. Um número complexo c faz parte do conjunto se, ao aplicar a equação iterativamente, os valores de z_n permanecem no limite determinado mesmo após várias iterações. Na prática, considere-se que a sequência diverge quando $z_n > 2$, pois se isso acontecer a sequência continuará crescendo indefinidamente.

Para gerar a representação visual do conjunto Mandelbrot, mapeia-se cada *pixel* de uma imagem para um número complexo c. As regiões do plano em que c pertence ao conjunto são então representadas visualmente.

Os valores de c são escolhidos em um retângulo do plano complexo, com a parte real no intervalo $[-2, 1]$ e a parte imaginária em $[-1.5, 1.5]$. A conversão das coordenadas dos *pixels* para coordenadas no plano complexo é feita pelas Equações 12 e 13, nas quais p_x e p_y representam as coordenadas do *pixel*, e $x_{textmin}$ e $x_{textmax}$ definem o domínio da parte real, enquanto $y_{textmin}$ e $y_{textmax}$ definem o domínio da parte imaginária.

$$c_x = x_{\text{min}} + \frac{p_x}{\text{largura da imagem}} \cdot (x_{\text{max}} - x_{\text{min}}) \tag{12}$$

$$c_y = y_{\text{min}} + \frac{p_y}{\text{altura da imagem}} \cdot (y_{\text{max}} - y_{\text{min}}) \tag{13}$$

Inserimos os valores c_x e c_y, nas iterações do Mandelbrot. Cada ponto da imagem é testado, e se a sequência z_n não divergir após um número máximo de iterações, o *pixel* corres-

pondente é colorido. A Figura 13 apresenta uma visualização típica do conjunto de Mandelbrot. O Algoritmo 8 (Debian Project, 2018) apresenta o cálculo do conjunto, e Algoritmo 9 é utilizado para desenhar a imagem do conjunto.

Algoritmo 8 – Cálculo do conjunto Mandelbrot

requer Número complexo c
1: $z \leftarrow 0 + 0i$
2: **para** $i = 0$ até 100, com passo 1 **faça**
3: $z \leftarrow z^2 + c$
4: **se** $|z| > 2.0$ **então**
5: **retorna** Falso
6: **finaliza se**
7: **finaliza para**
8: **retorna** Verdadeiro

Fonte: Adaptado do Debian Project (2018).

Algoritmo 9 – Geração da imagem do conjunto de Mandelbrot

requer Tamanho da imagem base n
1: $imageSize \leftarrow (n/8) \times 8$
2: $pixels \leftarrow$ vetor de tamanho $(imageSize \times imageSize)/8$, inicializado com 0
3: $scaleX \leftarrow (1.0 - (-2.0))/imageSize$
4: $scaleY \leftarrow (1.5 - (-1.5))/imageSize$
5: **para** $pixel = 0$ até $\texttt{tamanho}(pixels) - 1$ **faça**
6: $byte \leftarrow 0$
7: $byteRow \leftarrow imageSize/8$
8: $pixelColumn \leftarrow pixel \bmod byteRow$
9: $y \leftarrow pixel/byteRow$
10: **para** $bit = 0$ até 7 **faça**
11: $x \leftarrow pixelColumn \times 8 + bit$
12: $cx \leftarrow -2.0 + x \times scaleX$
13: $cy \leftarrow -1.5 + y \times scaleY$
14: **se** $\texttt{mandelbrot}(cx, cy) =$ Verdadeiro **então**
15: $byte \leftarrow byte \mid (1 \ll (7 - bit))$
16: **finaliza se**
17: **finaliza para**
18: $pixels[pixel] \leftarrow byte$
19: **finaliza para**

Fonte: Adaptado do Debian Project (2018).

Na implementação apresentada, duas otimizações foram aplicadas, no Algoritmo 9, para melhorar o desempenho do algoritmo. A primeira delas foi a adição de diretivas de paralelização, inseridas em seu laço mais externo, permitindo que diferentes regiões da imagem fossem processadas concorrentemente.

A segunda otimização foi a aplicação da técnica conhecida como *loop unrolling*, que consiste em transformar parte das iterações de um laço em instruções sequenciais explícitas. Essa técnica visa reduzir o *overhead* associado ao controle do laço, como verificações de condição e saltos, e pode beneficiar aspectos como *branch prediction* e uso mais eficiente do *pipeline* do processador.

4.2.7 PI de Monte Carlo

O método de Monte Carlo consiste em um conjunto de algoritmos estatísticos que utilizam amostragem aleatória para resolver problemas determinísticos ou estimar valores numéricos (Morettin; Bussab, 2010). Sua principal característica é a geração de entradas aleatórias num domínio previamente definido, seguido da aplicação de uma regra ou função para computar a saída, cujo resultado são então agregados para formar uma estimativa.

$$A_{\text{círculo}} = \pi r^2 = \pi \tag{14}$$

$$A_{\text{quadrado}} = 2 \times 2 = 4 \tag{15}$$

$$\frac{A_{\text{círculo}}}{A_{\text{quadrado}}} = \frac{\pi}{4} \tag{16}$$

$$\pi = 4 \times \frac{A_{\text{círculo}}}{A_{\text{quadrado}}} \tag{17}$$

No caso específico da estimativa de π, o método pode ser aplicado de maneira intuitiva. Considere um círculo de raio $r = 1$, inscrito em um quadrado de lado 2, conforme ilustrado na Figura 14. As áreas do círculo e do quadrado são dadas, respectivamente, pelas Equações 14 e 15. A razão entre essas duas áreas é apresentado na Equação 16. Por fim ao isolarmos o π, obtemos a Equação 17.

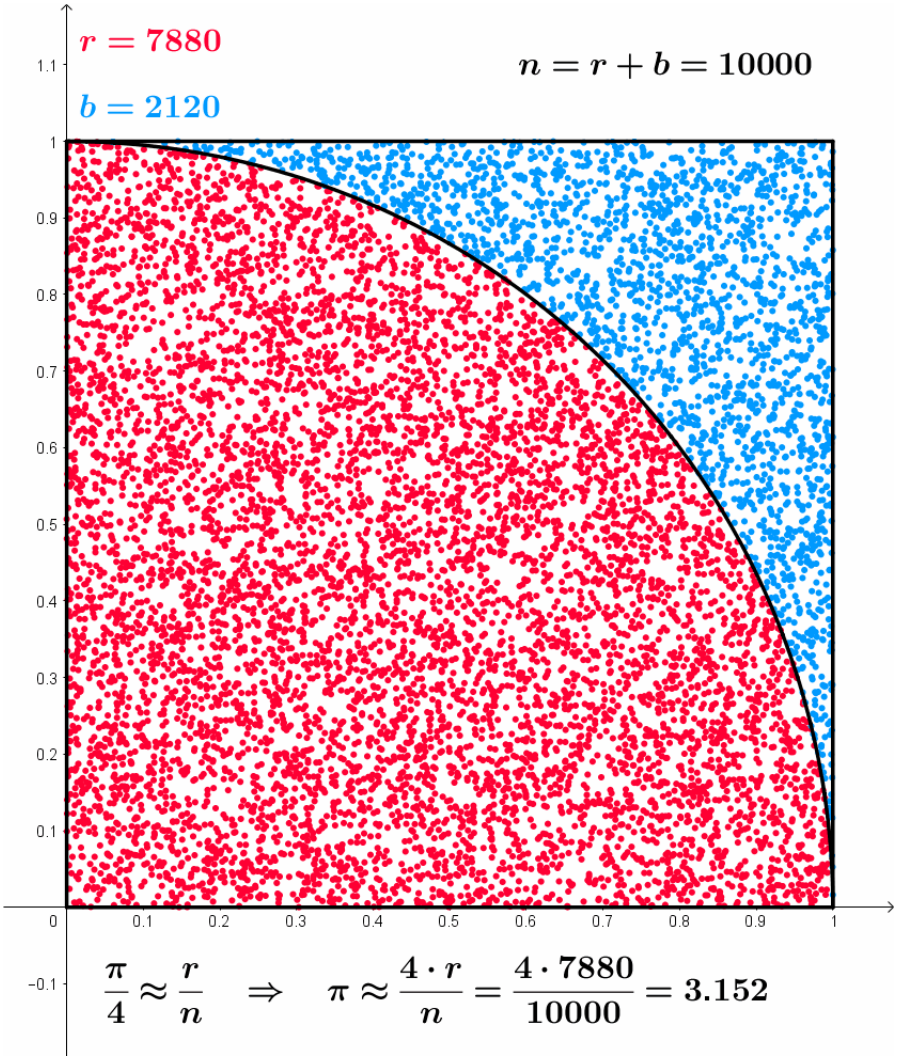
Algoritmo 10 – Cálculo de PI pelo método de Monte Carlo

requer Quantidade de iterações n
1: $hit \leftarrow 0$
2: $tid \leftarrow \texttt{thread_num}()$
3: $seedState \leftarrow \{tid, tid + 1\}$
4: **para** $i = 0$ até n, com passo 1 **faça**
5: $\quad x \leftarrow \texttt{random}(seedState)$
6: $\quad y \leftarrow \texttt{random}(seedState)$
7: $\quad$ **se** $(x \times x + y \times y) \leq 1$ **então**
8: $\quad\quad hit \leftarrow hit + 1$
9: $\quad$ **finaliza se**
10: **finaliza para**
11: **retorna** $(4.0 \times hit)/n$

Fonte: Autoria própria (2025).

A partir disso, é possível aplicar o método de Monte Carlo para estimar o valor de π. Como o círculo está inscrito no quadrado, o domínio de amostragem é o próprio quadrado. Gera-se então, um grande número de pontos aleatórios dentro do quadrado. Para cada ponto (x, y), calcula-se sua distância até a origem. Se a distância for menor ou igual a 1, o ponto está dentro do círculo.

Figura 14 – Método de Monte Carlo



Fonte: Kmhkmh (2023).

A estimativa de π é obtida pela razão entre o número de pontos que caíram dentro do círculo e o número total de pontos gerados, multiplicada por 4, como mostra a Equação 18, a Figura 14 apresenta uma ilustração desse processo de amostragem e da aproximação de π. O Algoritmo 10 apresenta a implementação dessas equações utilizada no *benchmark*.

$$\pi \approx 4 \times \frac{\text{pontos dentro do círculo}}{\text{total de pontos}} \tag{18}$$

Na implementação apresentada no Algoritmo 10, o laço principal foi paralelizado de forma que cada *thread* execute uma fração do espaço total de iterações. Outra otimização introduzida nessa implementação, foi na geração de números pseudoaleatórios com o algoritmo `xorshift128+`, escolhido por sua alta velocidade e boa qualidade estatística.

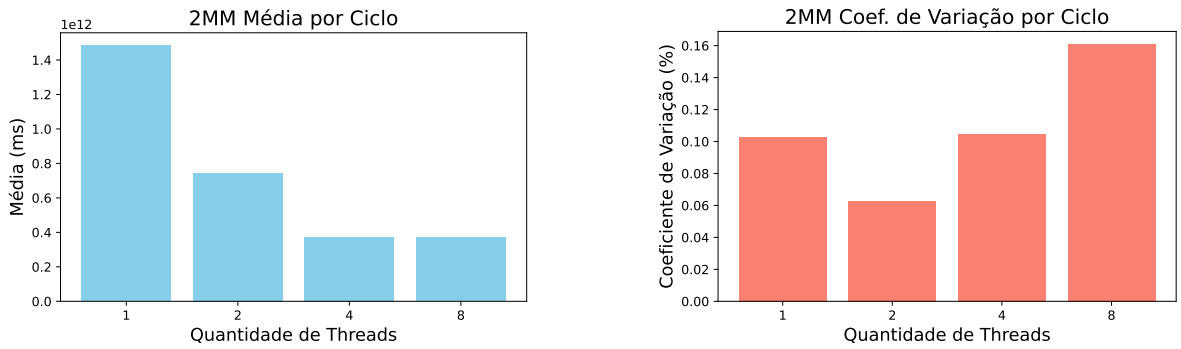
4.3 Análise de Desempenho

Visando identificar a forma mais adequada de medir o desempenho das aplicações, foram realizados experimentos com 2MM (sem *tiling*) e K-Means, utilizando 1, 2, 4 e 8 *threads*. Essas duas aplicações foram selecionadas porque apresentam tempos de execução mais elevados que as demais, evidenciando de maneira mais clara o impacto de cada modelo de medição.

As medições foram realizadas utilizando a ferramenta `perf`, que permite coletar métricas de baixo nível do *hardware* e do sistema operacional. Entre os principais eventos monitorados, destaca-se o número total de ciclos de CPU e o tempo total de execução das aplicações. As Figuras 15a e 15b apresentam, respectivamente, a média de ciclos e o coeficiente de variação da aplicação 2MM. Já as Figuras 17a e 17b mostram a média do tempo de execução e o respectivo coeficiente de variação. O mesmo pode ser observado em relação ao K-Means, nas Figuras 16a e 16b, que ilustram os resultados para os ciclos da aplicação, enquanto as Figuras 18a e 18b apresentam as medições referentes ao tempo de execução.

Figura 15 – Medição de ciclos na aplicação 2MM.

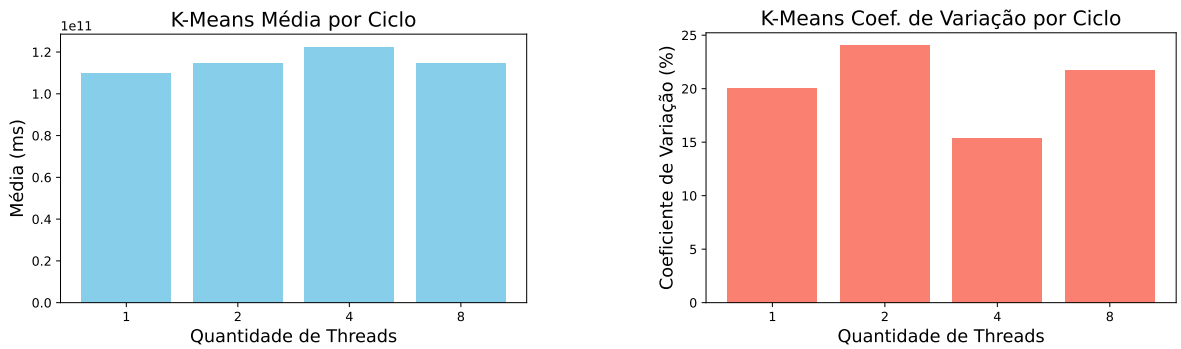
(a) Média de ciclos **(b) Coeficiente de variação dos ciclos**



Fonte: Autoria própria (2025).

Figura 16 – Medição de ciclos na aplicação K-Means.

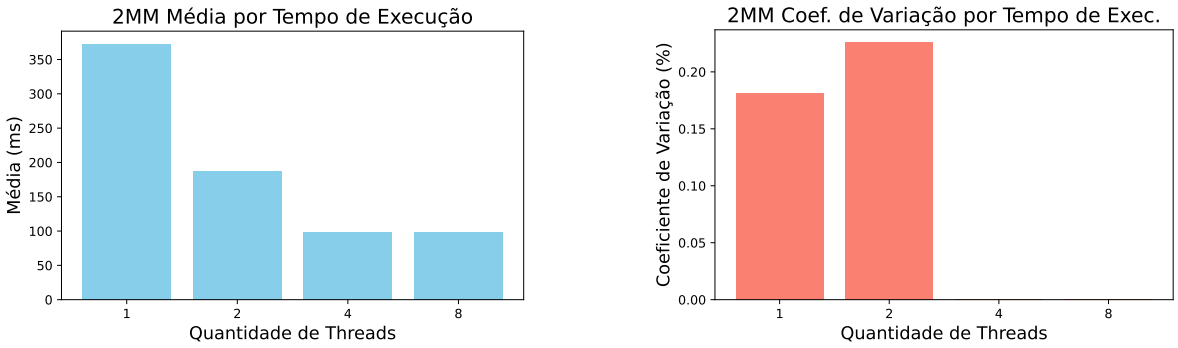
(a) Média de ciclos **(b) Coeficiente de variação dos ciclos**



Fonte: Autoria própria (2025).

Figura 17 – Medição de tempo na aplicação 2MM.

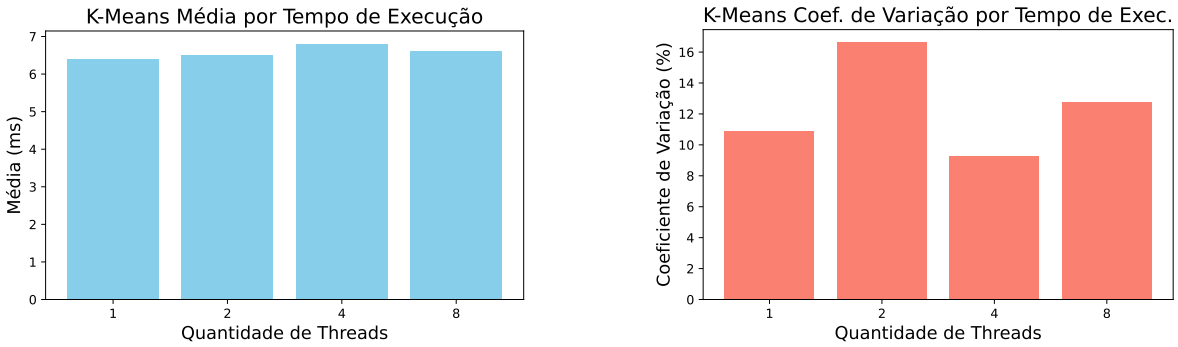
(a) Média de tempo **(b) Coeficiente de variação do tempo**



Fonte: Autoria própria (2025).

Figura 18 – Medição de tempo na aplicação K-Means.

(a) Média de tempo **(b) Coeficiente de variação do tempo**



Fonte: Autoria própria (2025).

Ao analisarmos as medições realizadas, observamos que a média tanto do tempo de execução quanto do número de ciclos apresentou padrões bastante similares em ambas as aplicações. No entanto, ao examinarmos o coeficiente de variação desses mesmos resultados, torna-se evidente que o tempo de execução sofreu um desvio significativamente menor entre as execuções quando comparado aos ciclos. Esse comportamento é especialmente claro no caso da aplicação 2MM, onde, com 4 e 8 threads, o coeficiente de variação do tempo de execução foi virtualmente nulo. Já no K-Means, o mesmo padrão se repete, por exemplo, com 1 *thread*, o coeficiente de variação dos ciclos atingiu cerca de 20%, enquanto no tempo de execução permaneceu próximo de 11%, uma diferença de quase $2\times$.

Diante dessa análise, optou-se por utilizar o tempo de execução como métrica principal para avaliar o desempenho, uma vez que apresentou maior estabilidade entre as execuções e refletiu de maneira mais consistente o comportamento das aplicações.

4.4 Resumo do conjunto de aplicações

Tabela 3 – Resumo das aplicações utilizadas nos experimentos

Aplicação	Fonte	Domínio	Métrica de Qualidade
2MM	PolyBench (Pouchet, 2011)	Álgebra Linear	SMAPE
Correlação	PolyBench (Pouchet, 2011)	Probabilidade e Estatística	SMAPE
Deriche	PolyBench (Pouchet, 2011)	Processamento de Imagem	SSIM
Jacobi 2D	PolyBench (Pouchet, 2011)	Solução Numérica	SMAPE
K-Means	Rodinia (Che *et al.*, 2009)	Aprendizado de Máquina	MCR
Mandelbrot	Debian Benchmarks Game (Debian Project, 2018)	Visualização Computacional	SMAPE
PI de Monte Carlo	Debian Benchmarks Game (Debian Project, 2018)	Probabilidade e Estatística	SMAPE

Fonte: Autoria própria (2025).

As aplicações selecionadas foram escolhidas por apresentarem diferentes padrões computacionais, possuírem suporte consistente à paralelização e incluírem etapas cujo resultado pode tolerar algum nível de aproximação. A Tabela 3 resume as aplicações utilizadas nos experimentos, indicando suas fontes, domínios e métricas de qualidade. Cada uma das aplicações selecionadas, serviram para avaliar como as técnicas de aproximação implementadas afetam o desempenho e a acurácia, considerando diferentes ambientes computacionais e padrões de paralelismo.

As métricas de qualidade foram definidas conforme a natureza da saída produzida por cada aplicação. Para aplicações com resultados numéricos contínuos, como 2MM, Correlação, Jacobi 2D e PI de Monte Carlo, adotou-se o SMAPE, pois essa métrica permite comparar valores aproximados e exatos mesmo quando variam de escala. No caso do K-Means, utilizou-se o MCR por ser uma métrica específica para avaliar a precisão da atribuição dos pontos aos *clusters*. Já na aplicação Deriche, cujo resultado é uma imagem filtrada, optou-se pelo *ssim*, que mede a similaridade estrutural entre as imagens.

4.5 Cronograma

A realização deste trabalho é organizada em partes, as quais são descritas a seguir:

1. Realizar uma revisão da literatura relacionada à CA, técnicas de aproximação e CP.

2. Revisar a infraestrutura do `OpenMP`.

3. Investigar a infraestrutura do LLVM.
4. Revisar aplicações de *benchmark* que podem ser aplicadas em sistemas aproximados.
5. Adaptar um *benchmark suite* para que ele se encaixe nas otimizações paralelas e aproximadas.
6. Testar e documentar as execuções dessas aplicações.
7. Implementar técnicas de aproximação paralela na infraestrutura do `OpenMP` integrada ao LLVM.
8. Testar e documentar as execuções do *benchmark* em conjunto com as novas técnicas apresentadas.
9. Investigar e avaliar o desempenho e a acurácia de cada uma das aplicações.
10. Documentar todos os procedimentos e resultados.
11. Escrita do trabalho de conclusão.
12. Defesa do projeto de Trabalho de Conclusão de Curso (TCC1).
13. Defesa do projeto de Trabalho de Conclusão de Curso (TCC2).

A Tabela 4 detalha o planejamento temporal para a execução de cada uma das atividades descritas.

Tabela 4 – Cronograma de execução das atividades

Atividades	Ago	Set	Out	Nov	Dez	Jan	Fev	Mar
1	X	X						
2		X	X					
3		X	X					
4			X	X				
5			X	X				
6			X	X				
7					X	X	X	
8						X	X	
9							X	X
10	X	X	X	X	X	X	X	X
11	X	X	X	X	X	X	X	X
12				X				
13								X

Fonte: Autoria própria (2025).

5 CONSIDERAÇÕES FINAIS

Este trabalho tem como objetivo melhorar o desempenho de aplicações de HPC por meio da aplicação combinada de técnicas de aproximação e paralelização. A busca por maior desempenho e eficiência energética surge devido às limitações enfrentadas na evolução do *hardware*, em decorrência do fim da Lei de Moore e da escala de Dennard.

Diante desse cenário, busca-se utilizar o paradigma da CA como uma alternativa para ampliar o desempenho e reduzir o consumo de recursos. Aplicações que não necessitam de resultados exatos e que demandam alta carga computacional podem se beneficiar das técnicas implementadas: perforação de laço, memoização aproximada, descarte de tarefas e relaxamento de ponto flutuante. Essas abordagens permitem ganhos expressivos de desempenho com impacto limitado na precisão, evitando modificações profundas no código-fonte.

Este trabalho propõe a criação de uma extensão para a ferramenta `OpenMP` dentro da infraestrutura do LLVM, de modo que esta passe a oferecer suporte nativo a diferentes técnicas de aproximação, integradas às diretivas de paralelização existentes. Com isso, pretende-se prover aos desenvolvedores uma interface simples e unificada para aplicar otimizações que resultem em maior desempenho, sem demandar grandes alterações na aplicação original.

Espera-se que, com a implementação dessas extensões, seja possível avaliar o impacto das técnicas aproximadas em aplicações reais, mensurando os ganhos de desempenho, as perdas de qualidade dos resultados e o custo de integração ao ambiente de execução. Para tal, será realizada a construção e análise de um conjunto de *benchmarks*, permitindo caracterizar os padrões de execução, a adequação de cada técnica e o equilíbrio entre desempenho e precisão.

REFERÊNCIAS

ADHIKARI, A. N. *et al.* Modeling large regions in proteins: Applications to loops, termini, and folding. *In*: PROTEIN SCIENCE. 21 n. 1., 2012. **Anais [...]** Wiley Online Library, 2012. p. 107–121. ISSN 0961-8368, 1469-896X. Disponível em: https://onlinelibrary.wiley.com/doi/10.1002/pro.767.

ALRAWAIS, A. Parallel programming models and paradigms: Openmp analysis. *In*: 2021 5TH INTERNATIONAL CONFERENCE ON COMPUTING METHODOLOGIES AND COMMUNICATION (ICCMC). 2021. **Anais [...]** IEEE, 2021. p. 1022–1029. Disponível em: http://dx.doi.org/10.1109/ICCMC51019.2021.9418401.

AXLER, S. **Linear Algebra Done Right**. Springer International Publishing, 2015. (Undergraduate Texts in Mathematics). ISBN 978-3-319-11079-0 978-3-319-11080-6. Disponível em: https://link.springer.com/10.1007/978-3-319-11080-6.

AYGUADÉ, E. *et al.* A proposal for task parallelism in openmp. *In*: SPRINGER. 2007. **Anais [...]** [*S.l.*], 2007. p. 1–12.

BAKOS, J. D. Memory optimization and video processing. *In*: **Embedded Systems**. Elsevier 2016. p. 147–185. ISBN 978-0-12-800342-8. Disponível em: https://linkinghub.elsevier.com/retrieve/pii/B9780128003428000043.

CHE, S. *et al.* Rodinia: A benchmark suite for heterogeneous computing. *In*: 2009 IEEE INTERNATIONAL SYMPOSIUM ON WORKLOAD CHARACTERIZATION (IISWC). 2009. **Anais [...]** [*S.l.*: *s.n.*], 2009. p. 44–54.

CHIPPA, V. K. *et al.* Analysis and characterization of inherent application resilience for approximate computing. *In*: PROCEEDINGS OF THE 50TH Annual Design Automation Conference. 2013. **Anais [...]** ACM, 2013. p. 1–9. ISBN 978-1-4503-2071-9. Disponível em: https://dl.acm.org/doi/10.1145/2463209.2488873.

DALLOO, A. M. *et al.* Approximate computing: Concepts, architectures, challenges, applications, and future directions. **IEEE Access**, Institute of Electrical and Electronics Engineers (IEEE) v. 12„ p. 146022–146088, 2024.

Debian Project. **Benchmarks Game — benchmarksgame-team.pages.debian.net**. [*S.l.*]: , 2018. https://benchmarksgame-team.pages.debian.net/benchmarksgame/performance/mandelbrot.html. Acesso em: 20 out. 2025.

DERICHE, R. Using Canny's criteria to derive a recursively implemented optimal edge detector, . v. 1, n. 2, p. 167–187, 1987. ISSN 0920-5691, 1573-1405. Disponível em: http://link.springer.com/10.1007/BF00123164.

DEVANEY, R. L. The mandelbrot set, the farey tree, and the fibonacci sequence. **The American Mathematical Monthly**, Informa UK Limited v. 106, n. 4, p. 289–302, abr. 1999. ISSN 1930-0972. Disponível em: http://dx.doi.org/10.1080/00029890.1999.12005046.

FABRíCIO, J. **Software and hardware interfaces for approximate memories: Interfaces software e hardware para memórias aproximadas**. 2022. Tese de Doutorado em Ciência da Computação 2022. Disponível em: https://repositorio.unicamp.br/Busca/Download?codigoArquivo=548558.

FABRíCIO, J. *et al.* AxRAM: A lightweight implicit interface for approximate data access, . v. 113„ p. 556–570, 2020. ISSN 0167739X. Disponível em: https://linkinghub.elsevier.com/retrieve/pii/S0167739X20301060.

FELZMANN, I. *et al.* Special Session: How much quality is enough quality? A case for acceptability in approximate designs. *In*: 2021 IEEE 39TH International Conference ON Computer Design (ICCD). 2021. **Anais [...]** IEEE, 2021. p. 5–8. ISBN 978-1-6654-3219-1. Disponível em: https://ieeexplore.ieee.org/document/9643638/.

FINK, Z. *et al.* HPAC-Offload: Accelerating HPC Applications with Portable Approximate Computing on the GPU. *In*: PROCEEDINGS OF THE International Conference FOR High Performance Computing, Networking, Storage AND Analysis. 2023. **Anais [...]** ACM, 2023. p. 1–14. ISBN 979-8-4007-0109-2. Disponível em: https://dl.acm.org/doi/10.1145/3581784.3607095.

FLYNN, M. J. Some Computer Organizations and Their Effectiveness, . C-21, n. 9, p. 948–960, 1972. ISSN 0018-9340. Disponível em: http://ieeexplore.ieee.org/document/5009071/.

GENTRYX. **File:2D von Neumann Stencil.svg - Wikimedia Commons — commons.wikimedia.org**. [*S.l.*]: , 2011. https://commons.wikimedia.org/wiki/File:2D_von_Neumann_Stencil.svg. Acesso em: 20 out. 2025.

GNU Project. **Optimize Options (Using the GNU Compiler Collection (GCC) 3.4.3)**. [*S.l.*]: , 2004. Online. Disponível em: https://gcc.gnu.org/onlinedocs/gcc-3.4.3/gcc/Optimize-Options.html. Acesso em: 17 nov. 2025.

GONçALVES, R. *et al.* Openmp is not as easy as it appears. *In*: . 2016, Koloa, Hawaii. **Anais [...]** Koloa, Hawaii: IEEE, 2016.

GRIGORIAN, B.; REINMAN, G. Accelerating Divergent Applications on SIMD Architectures Using Neural Networks, . v. 12, n. 1, p. 1–23, 2015. ISSN 1544-3566, 1544-3973. Disponível em: https://dl.acm.org/doi/10.1145/2717311.

HENNESSY, J.; PATTERSON, D. **Computer Architecture: A Quantitative Approach**. Elsevier Science, . (The Morgan Kaufmann Series in Computer Architecture and Design). ISBN 978-0-12-811905-1. Disponível em: https://books.google.com.br/books?id=MBQFuAEACAAJ.

HENNESSY, J. L.; PATTERSON, D. A. A new golden age for computer architecture. **Communications of the ACM**, Association for Computing Machinery (ACM) v. 62, n. 2, p. 48–60, jan. 2019. ISSN 1557-7317. Disponível em: http://dx.doi.org/10.1145/3282307.

HESSE, H. **Steppenwolf**. [*S.l.*]: Penguin Classics, 2025. (Penguin Classics). ISBN 9780143137825.

INCHEOL. **File:Kmeans wronginit2.PNG - Wikimedia Commons — commons.wikimedia.org**. [*S.l.*]: , 2015. https://commons.wikimedia.org/wiki/File:Kmeans_wronginit2.PNG. Acesso em: 20 out. 2025.

KMHKMH. **File:Pi monte carlo all.gif - Wikimedia Commons — commons.wikimedia.org**. [*S.l.*]: , 2023. https://commons.wikimedia.org/wiki/File:Pi_monte_carlo_all.gif. Acesso em: 20 out. 2025.

LASHGAR, A.; ATOOFIAN, E.; BANIASADI, A. Loop Perforation in OpenACC. *In*: 2018 IEEE Intl Conf ON Parallel & Distributed Processing WITH Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom). 2018. **Anais [...]** IEEE, 2018. p. 163–170. ISBN 978-1-7281-1141-4. Disponível em: https://ieeexplore.ieee.org/document/8672231/.

LEON, V. *et al.* Approximate Computing Survey, Part II: Application-Specific & Architectural Approximation Techniques and Applications, . Association for Computing Machinery (ACM) v. 57, n. 7, p. 1–36, 2025. ISSN 0360-0300, 1557-7341. Disponível em: https://dl.acm.org/doi/10.1145/3711683.

LEON, V. *et al.* Approximate Computing Survey, Part I: Terminology and Software & Hardware Approximation Techniques, . v. 57, n. 7, p. 1–36, 2025. ISSN 0360-0300, 1557-7341. Disponível em: https://dl.acm.org/doi/10.1145/3716845.

LI, S.; PARK, S.; MAHLKE, S. Sculptor: Flexible Approximation with Selective Dynamic Loop Perforation. *In*: PROCEEDINGS OF THE 2018 International Conference ON Supercomputing. 2018. **Anais [...]** ACM, 2018. p. 341–351. ISBN 978-1-4503-5783-8. Disponível em: https://dl.acm.org/doi/10.1145/3205289.3205317.

MACQUEEN, J. Some methods for classification and analysis of multivariate observations. *In*: PROCEEDINGS OF THE FIFTH BERKELEY SYMPOSIUM ON MATHEMATICAL STATISTICS AND PROBABILITY, VOLUME 1: STATISTICS. 1967, Berkeley, CA. **Anais [...]** Berkeley, CA: University of California Press, 1967. p. 281–297.

MATTSON, T. G.; HE, Y. H.; KONIGES, A. E. **The OpenMP common core**. [*S.l.*]: , 2019.

MICHIE, D. “memo” functions and machine learning. **Nature**, Nature Publishing Group UK London v. 218, n. 5136, p. 19–22, 1968.

MICROSOFT. **float_control pragma**. [*S.l.*]: , 2021. Online. Disponível em: https://learn.microsoft.com/en-us/cpp/preprocessor/float-control?view=msvc-170. Acesso em: 17 nov. 2025.

MITTAL, S. A Survey of Techniques for Approximate Computing, . v. 48, n. 4, p. 1–33, 2016. ISSN 0360-0300, 1557-7341. Disponível em: https://dl.acm.org/doi/10.1145/2893356.

MONNIAUX, D. The pitfalls of verifying floating-point computations. **ACM Transactions on Programming Languages and Systems**, v. 30, n. 3, p. 1–41, maio 2008. ISSN 0164-0925, 1558-4593.

MORETTIN, P. A.; BUSSAB, W. d. O. **Estatística básica**. 6. ed. rev. atual. ed. [*S.l.*]: Saraiva, 2010. ISBN 978-85-02-08177-2.

OLIVEIRA, J. B.; GONçALVES, R. A.; FILHO, J. F. Multithread Approximation: A new OpenMP construct. *In*: ANAIS DO XXV Simpósio EM Sistemas Computacionais DE Alto Desempenho (SSCAD 2024). 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 372–383. Disponível em: https://sol.sbc.org.br/index.php/sscad/article/view/31012.

OLIVEIRA, J. B.; GONçALVES, R. A.; FILHO, J. F. Openmp em direção à aproximação: Loop perforation e multithreading. *In*: ANAIS DA XV ESCOLA REGIONAL DE ALTO DESEMPENHO DE SãO PAULO (ERAD-SP 2024). 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 49–52. Disponível em: https://sol.sbc.org.br/index.php/eradsp/article/view/28732.

OLIVEIRA, J. V. B. D.; GONçALVES, R. A.; FILHO, J. F. Anotação de código de alto nível para aproximação de sincronização e dados. *In*: ANAIS DO XIV SEMINáRIO DE EXTENSãO E INOVAçãO & XXIX SEMINáRIO DE INICIAçãO CIENTíFICA E TECNOLóGICA DA UTFPR – SEI/SICITE. 2024, Francisco Beltrão, PR. **Anais [...]** Francisco Beltrão, PR: UTFPR, 2024. Acesso em: 17/11/2025. Disponível em: https://www.even3.com.br/anais/seisicite2024/965072-ANOTACAO-DE-CODIGO-DE-ALTO-NIVEL-PARA-APROXIMACAO-DE-SINCRONIZACAO-E-DAD

OPENMP. **Openmp API Specification**. [*S.l.*]: , 2018. Disponível em: https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5.0.pdf. Acesso em: 17 nov. 2025.

OpenMP Architecture Review Board. **OpenMP Application Programming Interface**. [*S.l.*], 2021. Disponível em: http://www.openmp.org/.

PARASYRIS, K. *et al.* HPAC: Evaluating approximate computing techniques on HPC OpenMP applications. *In*: PROCEEDINGS OF THE International Conference FOR High Performance Computing, Networking, Storage AND Analysis. 2021. **Anais [...]** ACM, 2021. p. 1–14. ISBN 978-1-4503-8442-1. Disponível em: https://dl.acm.org/doi/10.1145/3458817.3476216.

POUCHET, L.-N. **polybench — sourceforge.net**. [*S.l.*]: , 2011. https://sourceforge.net/projects/polybench/. Acesso em: 13 out. 2025.

REIS, L.; WANNER, L. Functional approximation and approximate parallelization with the accept compiler. *In*: . 2021. **Anais [...]** [*S.l.*: *s.n.*], 2021. p. 188–197.

REIS, L.; WANNER, L.; RIGO, S. Towards Just-In-Time Software Approximations. *In*: ANAIS DO XXV Simpósio EM Sistemas Computacionais DE Alto Desempenho (SSCAD 2024). 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 360–371. Disponível em: https://sol.sbc.org.br/index.php/sscad/article/view/31011.

SAMPSON, A. *et al.* Accept: A programmer-guided compiler framework for practical approximate computing. **University of Washington Technical Report UW-CSE-15-01**, v. 1, n. 2, p. 1–14, 2015.

SIDIROGLOU-DOUSKOS, S. *et al.* Managing performance vs. accuracy trade-offs with loop perforation. *In*: PROCEEDINGS OF THE 19TH ACM SIGSOFT SYMPOSIUM AND THE 13TH European CONFERENCE ON Foundations OF SOFTWARE ENGINEERING. 2011. **Anais [...]** ACM, 2011. p. 124–134. ISBN 978-1-4503-0443-6. Disponível em: https://dl.acm.org/doi/10.1145/2025113.2025133.

STATISTICS, L. **File:Pearson Correlation Coefficient and associated scatterplots.png - Wikimedia Commons — commons.wikimedia.org**. [*S.l.*]: , 2019. https://commons.wikimedia.org/wiki/File:Pearson_Correlation_Coefficient_and_associated_scatterplots.png. Acesso em: 20 out. 2025.

SZéP, G. **Handbook of Mathematics**. 5th ed. ed. [*S.l.*]: Springer, 2007. ISBN 978-3-540-72121-5.

TANENBAUM, A. S. **Modern Operating Systems**. Fourth edition. [*S.l.*]: Pearson, . ISBN 978-0-13-359162-0.

The LLVM Project. **Clang Compiler User's Manual**. [*S.l.*]: , 2025. Online. Disponível em: https://clang.llvm.org/docs/UsersManual.html#cmdoption-ffast-math. Acesso em: 17 nov. 2025.

TZIANTZIOULIS, G.; HARDAVELLAS, N.; CAMPANONI, S. Temporal Approximate Function Memoization. **IEEE Micro**, v. 38, n. 4, p. 60–70, jul. 2018. ISSN 0272-1732, 1937-4143.

VASSILIADIS, V. *et al.* A programming model and runtime system for significance-aware energy-efficient computing. **ACM SIGPLAN Notices**, ACM New York, NY, USA v. 50, n. 8, p. 275–276, 2015.

XU, Q.; MYTKOWICZ, T.; KIM, N. S. Approximate Computing: A Survey. **IEEE Design and Test**, v. 33, n. 1, p. 8–22, feb 2016. ISSN 21682356. Disponível em: http://ieeexplore.ieee.org/document/7348659/.